https://pubsonline.informs.org/journal/mnsc

MANAGEMENT SCIENCE
Vol. 68, No. 10, October 2022, pp. 7497–7517
ISSN 0025-1909 (print), ISSN 1526-5501 (online)

Optimal Picking Policies in E-Commerce Warehouses

Maximilian Schiffer,[a,b,*] Nils Boysen,[c] Patrick S. Klein,[a] Gilbert Laporte,[d,e] Marco Pavone[f]

[a] School of Management, Technical University of Munich, 80333 Munich, Germany; [b] Munich Data Science Institute, Technical University of Munich, 80333 Munich, Germany; [c] Operations Management, Friedrich Schiller Universität Jena, 07743 Jena, Germany; [d] HEC Montréal, Montréal, Quebec H3T 2A7, Canada; [e] School of Management, University of Bath, Bath BA2 7AY, United Kingdom; [f] Department of Aeronautics and Astronautics, Stanford University, Stanford, California 94035
*Corresponding author

Contact: schiffer@tum.de, https://orcid.org/0000-0003-2682-4975 (MS); nils.boysen@uni-jena.de, https://orcid.org/0000-0002-1681-4856 (NB); patrick.s.klein@tum.de, https://orcid.org/0000-0002-2851-6047 (PSK); gilbert.laporte@cirrelt.ca, https://orcid.org/0000-0003-0869-1507 (GL); pavone@stanford.edu, https://orcid.org/0000-0002-0206-4337 (MP)

Received: August 1, 2018
Revised: August 23, 2019; July 19, 2020; May 31, 2021; October 7, 2021
Accepted: October 30, 2021
Published Online in Articles in Advance: March 24, 2022

https://doi.org/10.1287/mnsc.2021.4275

Abstract. In e-commerce warehouses, online retailers increase their efficiency by using a mixed-shelves (or scattered storage) concept, where unit loads are purposefully broken down into single items, which are individually stored in multiple locations. Irrespective of the stock keeping units a customer jointly orders, this storage strategy increases the likelihood that somewhere in the warehouse the items of the requested stock keeping units will be in close vicinity, which may significantly reduce an order picker's unproductive walking time. This paper optimizes picker routing through such mixed-shelves warehouses. Specifically, we introduce a generic exact algorithmic framework that covers a multitude of picking policies, independently of the underlying picking zone layout, and is suitable for real-time applications. This framework embeds a bidirectional layered graph algorithm that provides the best known performance for the simple picking problem with a single depot and no further attributes. We compare three different real-world e-commerce warehouse settings that differ slightly in their application of scattered storage and in their picking policies. Based on these, we derive additional layouts and settings that yield further managerial insights. Our results reveal that the right combination of drop-off points, dynamic batching, the utilization of picking carts, and the picking zone layout can greatly improve the picking performance. In particular, some combinations of policies yield efficiency increases of more than 30% compared with standard policies currently used in practice.

History: Accepted by Chung Piaw Teo, optimization.
Supplemental Material: The online appendices are available at https://doi.org/10.1287/mnsc.2021.4275.

Keywords: mixed-shelves warehouse • order picking policies • picking zone layout • dynamic programming

1. Introduction

A paradigm change in the retail business, caused by rapidly growing e-commerce (Cui et al. 2019), makes today's warehouses the focus of efficient operations in business-to-customer (B2C) markets. Although warehouses have traditionally been viewed as remote, negligible, and cost-efficient planning components, today's warehouses have evolved into technology-enriched logistics facilities that play a key role in retail supply chains. For companies of any significant size, the warehouse of today and of the future performs several of the functions that conventional stores have previously managed. Consequently, firms' attention to innovative concepts and efficient warehouse operations is steadily increasing, as warehouses are seen as a key success factor in the highly competitive e-commerce market. In these warehouses, efficient daily operations are central to running a profitable e-commerce business and to fulfilling customer expectations, such as same-day deliveries, such that efficiently routing pickers constitutes a crucial determinant of profitability.

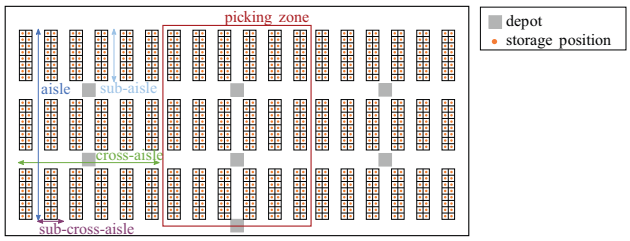
In e-commerce warehouses, two fundamentally different concepts dominate: robotized parts-to-picker warehouses, where (KIVA) robots bring man-high shelves to stationary pickers (Azadeh et al. 2018), and human-operated picker-to-parts warehouses in which pickers traditionally walk through the shelves and collect stock keeping units (SKUs). These two concepts entail a trade-off between investment costs, scalability, and efficiency. *Parts-to-picker warehouses* have high investment costs that increase significantly with the number of robots but are highly efficient when operated at their maximum throughput. They are hardly scalable to rare events during which the workload increases significantly, for example, on Black Friday or for Cyber Monday sales. *Picker-to-parts warehouses* have low investment costs and are less efficient than robotized systems. In contrast, these

warehouses allow—by hiring additional temporary staff—the necessary flexibility in peak times. Hence, major players in the industry agree that picker-to-parts warehouses should remain in parallel to robotized warehouses to provide flexibility.

With standardized third-party robotized warehouse solutions across companies, increasing the efficiency of picker-to-parts warehouses becomes even more crucial for online retailers who vie to derive a competitive advantage. In traditional picker-to-parts warehouses (with unit loads kept together), the storage assignment plans result in excessive unproductive walking for the pickers, corresponding to as much as 50% of their working hours (De Koster et al. 2007). Many online retailers in the B2C segment apply the mixed-shelves or scattered storage concept (Weidinger and Boysen 2018) to increase the efficiency of such warehouses. Under this storage assignment policy, incoming loads of SKUs are purposefully broken down into single items scattered over all parts of the warehouse, thus increasing the likelihood that, given the volatile demands of the final customers, items finally ordered together will be in close vicinity somewhere in the warehouse (Boysen et al. 2019a). However, competitive routing algorithms that determine picker paths to collect SKUs remain crucial to an efficient exploitation of the scattered storage in e-commerce warehouses and to the reduction of unproductive picker walking time. In particular, large e-commerce warehouses subdivide their order fulfillment process into three stages, that is, order picking, intermediate storing of picked bins, and order accumulation and packing (see Section 2.1). Because the latter two stages are rather standardized and can largely be automated, order picking remains the most crucial and labor-intensive stage and is, therefore, the focus of this paper.

Today's e-commerce warehouses consist of picking zones, that is, limited blocks of shelves in which pickers operate (see Figure 1). These zones have a rectilinear layout, which means that the pickers always move according to a Manhattan metric, with shelves placed along parallel aisles and one or several drop-off points at which pickers can hand over items. Online retailers apply batch picking, that is, they unify multiple picking orders into a single picklist (Boysen et al. 2019a)

Figure 1. (Color online) Example of the Storage Area of an E-Commerce Warehouse and a Picking Zone



that is then processed by a picker in a dedicated zone. The picker starts with a manual picking cart and a picklist at a drop-off point and returns to it once the picklist is completed. Despite the regular rectilinear layout and the rather simple picking process, picking zones may differ in the number of cross-aisles and aisles they contain. Although most companies rely on the scattered storage principle and agree on efficient picking as a key to successful operations, there is no consensus on which general attributes a good picking policy and a picking zone layout should possess. This paper focuses on four main attributes that have been identified as most frequently implemented in practice in the recent survey of Boysen et al. (2019a):

i. *a variable multiblock layout* that differs in terms of the number of aisles and cross-aisles;

ii. *multiple drop-off points* that increase the number of points in a picking zone where completed bins can be handed over to a central conveyor system;

iii. *dynamic batching policies* that allow to drop a finished picklist early while processing several picklists in parallel by equipping the picking cart with multiple bins; and

iv. *cartless subtours* that allow pickers to intermediately park their clumsy cart in order to pick a few items on a subtour much faster.

Several site visits that we have made in European e-commerce warehouses have confirmed that these policies and layout options are applied in various combinations (see Section 2.2) and benefits can be achieved through optimization. However, performance gains come at the price of additional investments, which may differ for each setting. Hence, we evaluate the performance impact of the picking policy and layout options, which allows us to quantify each measure's improvement potential without relating it to varying and hence hardly assessable investment costs. To set our study apart from recent research, we first briefly review the related literature in Section 1.1 before further detailing the contributions of our work in Section 1.2.

1.1. Literature Review

Efficient order processing in warehouses and distribution centers has been vividly discussed in recent years. For an overview of the rich body of literature on traditional picker-to-parts warehouses, modern robotized warehouses, and the peculiarities of warehousing in the e-commerce era, we refer to the surveys of De Koster et al. (2007), Azadeh et al. (2018), and Boysen et al. (2019a). In the following, we only survey the picker routing literature related to the planning tasks outlined above.

Picker routing starting and ending at a central drop-off point corresponds to the traveling salesman problem (TSP). In their seminal paper, Ratliff and Rosenthal (1983)

showed that in single-block warehouses consisting of parallel aisles with cross-aisles at the front and back, the particular structure of the distance matrix allows solving the resulting TSP in polynomial time by dynamic programming. Roodbergen and De Koster (2001) extended this approach to two-block warehouses with an additional middle aisle, and De Koster and Van der Poort (1998) considered a handover of complete orders at the start and end of each aisle. Recently, Pansart et al. (2018) generalized the approach of Ratliff and Rosenthal (1983) to multiblock warehouses using the polynomial-time algorithm of Cambazard and Catusse (2018) for rectilinear Steiner tree problems in the plane. As can be seen, exact algorithms for the picker routing problem are rare and limited to special cases.

Several heuristics have been proposed for the picker routing problem itself and for some of its variants, for example, picker routing combined with selecting pick positions or with zoning and batching policies. For a detailed overview of these heuristics, we refer the reader to De Koster et al. (2007). While fundamental work including performance analysis was carried out by Hall (1993) and Petersen (1997), De Koster and Van der Poort (1998) provided comparisons between exact and heuristic algorithms for specific layouts. More recent approximate algorithms adapt competitive TSP heuristics to the picker routing problem (Theys et al. 2010). Gue et al. (2006), Hong et al. (2012a), and Chen et al. (2013) have all focused on blocking aspects in narrow aisles. Given the dedicated zoning policy and layout of modern e-commerce warehouses, this blocking issue is negligible nowadays. Only one publication (Goetschalckx and Ratliff 1988) exists on pickers with different travel modes, that is, pickers who are allowed to park their cart and perform cartless subtours on foot. However, that paper assumes that a picker moves the vehicle along a single aisle but starting and stopping the vehicle to retrieve items on foot consumes additional time. As can be seen, no heuristic exists that captures the generic nature of our problem.

Weidinger et al. (2019) showed that the integrated selection of pick positions for a specific demand in a mixed-shelves warehouse makes the picker routing problem strongly NP-hard, even in elementary block-shaped warehouses with parallel aisles. Algorithms for this problem class were proposed by Daniels et al. (1998), Weidinger (2018), and Weidinger et al. (2019). These authors decompose the problem by fixing the pick positions' selection and treating the second-stage routing problem heuristically. Overviews of zoning and batching policies can be found in De Koster et al. (2007) and Boysen et al. (2019a). Recent contributions to this field by Bozer and Kile (2008), Henn and Wäscher (2012), Hong et al. (2012b), and Zulj et al. (2018) have focused on heuristics for a static batching policy. In practice, the two planning tasks of selecting pick positions and batching and zoning are decoupled from the picker routing decision. Recently, Weidinger (2018) gave proof to this common practice and showed that solutions derived in this fashion yield optimality gaps well below one percent compared with integrated planning. Hence, we exclude these problems from our real-world study and assume respective decisions to be made at an upstream decision stage.

Concluding, no exact or heuristic algorithm exists that captures the generic nature of our problem, namely, (i) *a variable multiblock layout*, (ii) *multiple drop-off points*, (iii) *dynamic batching policies*, and (iv) *cartless subtours*. In particular, requirements (ii)–(iv) have not yet been incorporated in any exact or heuristic algorithm to the best of our knowledge.

1.2. Contributions

In this paper, we close the gap in the literature outlined in Section 1.1 by developing the first generic exact algorithm that can handle the four features just mentioned. Its computational requirements must be sufficiently low to allow for real-time application in practical settings. Given this, the methodological and managerial contribution of this paper is threefold. First, we develop an exact algorithm for picker routing capable of capturing the four main attributes identified by Boysen et al. (2019a). Second, we provide additional preprocessing techniques that allow the real-time use of this algorithm in real-world applications. Third, based on this methodological framework, we conduct different experiments to benchmark several picking policies and layout options on realistic data sets extracted from three corporate case studies.

Our algorithm provides a new state of the art for picker routing problems. In particular, we outperform the previously best known exact algorithm (Pansart et al. 2018) for single picklists in multiblock warehouses in terms of computational time; we obtain better scalability for large-size instances with up to 11 cross-aisles. Besides, our algorithm is significantly more general as it can solve picker routing problems with multiple picklists, dynamic batching strategies, multiple drop-off points, and cartless subtours. We use this algorithm to identify the benefits of all considered picking policy design options utilizing a full-factorial experiment. Specifically, we show that the level of picklist dispersion, that is, a picklist's warehousing area that results from the selection of more or less dispersed picking positions during upper level planning tasks, strongly influences strategic design and operational decisions.

1. At the strategic level, we show that increasing the number of cross-aisles can yield efficiency increases of more than 20% in the case of dispersed picklists. However, for clustered picklists, the reduction potential is much smaller; increasing the number of cross-aisles

may even increase the total picker walking time. The deployment of multiple drop-off points shows decreasing marginal utilities for large numbers of drop-off points.

2. At the operational level, we show that dynamic batching yields significant efficiency improvements of up to 9% for clustered picklists but has only negligible effects in the case of dispersed picklists. In contrast, cartless subtours yield efficiency improvements of up to 11% for dispersed picklists but have a negligible effect in the case of clustered picklists.

3. Combining the right operational and strategic design decisions can lead to efficiency improvements of up to 31% for clustered picklists and of up to 11% for dispersed picklists.

1.3. Organization of the Paper

The remainder of this paper is structured as follows. Section 2 further details the fulfillment process in an e-commerce warehouse, details the settings of picking policies we have observed in practice, and provides a formal definition of the planning problems at hand. Section 3 develops our algorithm. We present extensive computational results in Section 4. Section 5 concludes this paper and provides several managerial insights. All proposition proofs as well as additional material are provided in an online appendix.

2. Problem Description

This section introduces our planning problem. We first detail the order fulfillment process in e-commerce warehouses to show the integration of our planning problem into daily operations. We then discuss the settings of picking policies that we have observed in practice before formally introducing the generalized order picking problem (GOPP).

2.1. Order Fulfillment Process

For conciseness, we exclude the ingoing flow of the warehouse and focus on the outgoing flow. Once items are stored and scattered around the warehouse, the order fulfillment process in a mixed-shelves warehouse consists of three basic steps.

2.1.1. Order Picking. First, customer orders must be picked from their respective storage positions. Human pickers, each equipped with a small picking cart that carries small bins, collect items into these bins. A picker receives empty bins and picklists, each associated with a bin, at a drop-off point, strides (directed by a hand-held scanner or a pick-by-voice system) through the warehouse, collects the requested items from the shelves, and puts them into their respective bin on the cart. Once one or multiple bins are completed, the picker hands the bins over to the central conveyor system at a drop-off point and starts processing the next picklists. This process is typically executed under a zoning and batching policy (De Koster et al. 2007). Instead of storing all items in a single monolithic area, online retailers subdivide their vast shop floors into multiple zones to allow for parallel processing of customer orders. A picker operates exclusively in one of these zones and only picks the part of an order stored in the assigned zone. By parallelizing the picking process in this fashion, order pickers traverse smaller areas of the warehouses to reduce their unproductive walking time. A further reduction of unproductive walking time can be achieved by unifying multiple orders into batches that are jointly retrieved on the picker's tour through the zone. This results in bins filled with partial orders for multiple customers.

In e-commerce, customer orders have different priorities and varying deadlines as some customers may participate in priority delivery programs. Directly considering these priorities leads to unrealistically long planning horizons of several hours that may even exceed the time needed to handle the orders known beforehand. In practice, warehouse operators consider the priority of orders in line with the order batching and picklist sequencing at an upstream planning stage before giving a picklist sequence, which comprises the most urgent orders for a short planning horizon, to a picker. This sorted list sequence comprises the picklists for the most urgent orders. The order waves that are not yet included in this planning horizon are more likely to be timely processed if the current set of picklists is assembled as fast as possible.

2.1.2. Intermediate Storing of Picked Bins. Once handed over to the central conveying system at a drop-off point, these bins are intermediately stored until all those belonging to the same wave of customer orders have arrived from all zones. Different storage devices exist. Some operators apply lane-based systems where bins of a wave are channeled in a conveyor queue. Others apply automated storage and retrieval systems, where bins are stored in crane-operated aisles of high-bay storage racks (Boysen et al. 2018). In smaller warehouses, loop-based systems where bins circle until a wave is complete may be applied (Gallien and Weber 2010).

2.1.3. Order Accumulation and Packing. Finally, a wave of bins released from intermediate storage arrives in the order accumulation area. Here, the incoming items are sorted according to customer orders. Some operators apply so-called put walls for this task, where human logistics workers place items on small shelves, each temporarily dedicated to a specific customer order (Boysen et al. 2019b). On the other side of the wall, packers receive completed orders, pack them into their shipping cartons, and forward

them via conveyors to their dedicated trailers. Other warehouses apply automated belt sorters, for example, tilt-tray or cross-belt sorters, where items are collected in packing stations arranged along the belt (Boysen et al. 2018).

Both *intermediate storing* and *packing* are process steps that can easily be standardized and organized efficiently. However, *order picking* includes unpredictable components because it depends on incoming customer orders and requires the largest fraction of the workforce. Therefore, an efficient routing algorithm plays a crucial role in ensuring efficient order fulfillment.

The inputs needed for creating route plans are the picklists, a picklist sequence, and the assignment of SKUs to picklists. Integrated problems that combine order batching or sequencing with picker routing have been discussed (Weidinger et al. 2019), but while they provide challenging research questions, their solutions are less relevant in practice. Indeed, operators typically solve the batching and sequencing problems at an upper level, decoupled from the picker routing, for the following reason: both problems heavily depend on the incoming orders' priorities. Those orders with the closest due date of their associated outbound trucks are selected as the wave of orders to be processed next. Therefore, all orders of the current wave are urgent, and there is no need to further differentiate between their priorities during order picking. Recent research has corroborated this common practice. Weidinger (2018) has shown that in practice selecting the shelves from which each item is to be picked in a scattered storage warehouse can be solved independently of the picker routing problem. Indeed, selecting shelves such that the current picklist's warehousing area is minimal and handing these pick positions over to picker routing leads to near-optimal solutions with optimality gaps well below one percent (Weidinger 2018).

2.2. Picking Policies

In this paper, we develop a generalized exact algorithm for picker routing in e-commerce warehouses. We create different settings based on different order-picking policies that we have observed in practice in mixed-shelves warehouses.

2.2.1. Amazon Europe. In Amazon's distribution centers in Bad Hersfeld (Germany) and Poznan (Poland), we have observed the following setting. Amazon uses multiple drop-off points to a central conveyor system to hand over completed bins. Pickers often pass by these local drop-off points during a tour, which offers the additional flexibility of dynamic batching. Instead of picking batch after batch with intermediate returns to a central drop-off point in a static manner, pickers can hand over just a subset of completed bins at a local drop-off point and retrieve new empty bins for subsequent orders. Thus, Amazon's picker routing procedure additionally considers the dynamic batching of an order set by inserting visits at different drop-off points in the tour where the picker hands over subsets of completed bins and starts handling new orders.

2.2.2. Hermes Group. This is Germany's second-largest postal service provider. Its distribution center in Haldensleben (Germany) operates full-service order fulfillment for one of Europe's largest online and catalog fashion retailers. Hermes's order picking process is as follows. Each picker is equipped with a picking cart with up to four bins, each having a capacity of about 20–30 items. Pickers operate in fixed zones, each having a single drop-off point where new bins and the next picklist are retrieved; finally, completed bins are handed over to the central conveyor system. Hence, each picker tour starts and ends at a central drop-off point; for a given batch of (four) picking orders, one seeks a tour through the respective zone of the warehouse, such that all items on the picklist can be retrieved, and the length of the tour is minimal.

2.2.3. Zalando. The online fashion retailer Zalando uses a different system at its distribution center in Erfurt (Germany). It applies static batching with a single drop-off point in each picking zone. Because the bins are relatively large and heavy when filled, picking carts are clumsy and inflexible. Hence, the pickers are much faster without the cart. Thus, a picker may park his or her cart, pick up to six items from close-by shelves, and carry them back to the cart. An optimization procedure for the Zalando case must determine whether a walk between two successive visits of a picker tour should be executed with or without a cart. This decision has to consider the limited capacity of items a picker can carry on his or her arms and the varying walking speeds with and without a cart.

Based on these observations, we classify picking policies according to three characteristics: (i) *static versus dynamic batching*, (ii) *single versus multiple drop-off points*, and (iii) *single versus multiple travel modes (i.e., cartless subtours)*. Certain combinations of these characteristics reflect the real-world picking policies detailed above. In our computational studies, we aim at a full factorial design to evaluate each degree of freedom's potential in designing picking policies. Hence, we add missing policies even if we have not yet observed them in practice. Table 1 shows the different picking policies with increasing degrees of freedom from left to right. As can be seen, the Amazon (A) policy already covers most degrees of freedom, whereas the Hermes (H) and the Zalando (Z) policy remain at the bottom line with respect to degrees of freedom.

Table 1. Picking Policies Resulting from Different Attributes

Policy	i (H)	ii (Z)	iii	iv	v	vi	vii (A)	viii
Static (SB) / dynamic (DB) batching	SB	SB	SB	SB	DB	DB	DB	DB
Single (SD) / multiple (MD) drop-off points	SD	SD	MD	MD	SD	SD	MD	MD
No cartless subtours (NS) / cartless subtours (CS)	NS	CS	NS	CS	NS	CS	NS	CS

Note. A-Amazon; H-Hermes; Z-Zalando.

2.3. A Generalized Order Picking Problem

Given the multitude of picking policies described in Section 2.2, we now formally introduce the GOPP.

2.3.1. Solution Representation and Notation. To represent a solution Π, we model positions in the picking zone (Figure 1) as vertices, such that a vertex is either a drop-off point ($c \in \mathcal{D}$), a crossing between an aisle and a cross-aisle ($c \in \mathcal{C}$), or a storage position ($c \in \mathcal{P}$) that contains varying amounts of different SKUs. We note that a solution will include only a subset of storage position vertices $\mathcal{P}' \subseteq \mathcal{P}$ that correspond to positions where items have to be picked. Analogously, only drop-off point vertices and cross-aisle vertices necessary to complete a tour are included in the respective solution. At the operational level, an ordered set $\mathcal{L}$ of picklists l denotes which position $c \in \mathcal{P}$ must be visited to pick up SKUs. Recall that the size and pick-up position of a given SKU is decided at a preceding planning level to efficiently split customer orders into picklists and avoid double picking by competing pickers. Hence, we represent a single picklist $l = \{c_1, \ldots, c_n\}$ as a finite set of n storage positions that must be visited. Across multiple picklists, n can vary as, for example, prioritized lists might be designed with fewer items on an upper planning level.

Besides, we use the set $\mathcal{L}_i^{\mathrm{a}}$ to track active picklists at each position i of a route. Depending on the picking policy, a picker processes a single picklist or multiple picklists in parallel such that $|\mathcal{L}_i^{\mathrm{a}}|$ is limited. Each picklist must be collected in a separate bin. Thus, we can measure the *picking cart capacity* κ, that is, the number of possible parallel-processed picklists, by the number of bins. In practice, standardized bins are a prerequisite for fail-safe transport on picking carts and conveyors. Note that our assumption remains valid even for cases with differently sized bins because the upstream planning stage avoids conflicting bins while defining the sequence of picklists.

A picker is accompanied by a cart equipped with standardized bins to collect orders. For certain picking policies, the picker may temporarily leave his or her cart to pick a certain number of SKUs during a *cartless subtour*.

Definition 1 (Cartless Subtour). A cartless subtour of the picker starts at the i^{th} stop of the tour where the picker leaves his or her cart and ends at stop k where he or she picks up the cart again at the same position such that $c_i = c_k$. In addition, a cartless subtour is limited to a certain number of items K, that is, the maximum number of items a picker can carry without cart support. A picker may start a subtour at the end of or at any position in a subaisle. A subtour may not span across more than one subaisle.

A solution Π is an ordered set that defines the successive visits of the picker in $\mathcal{V} = \mathcal{D} \cup \mathcal{C} \cup \mathcal{P}$. Each element of Π is a pair $\pi_i = (c_i, \mathcal{L}_i^{\mathrm{a}})$. The pair $\pi_0 = (c_0, \mathcal{L}_0^{\mathrm{a}})$ at the very first sequence position refers to the initial position of the picker such that c_0 is fixed to the picker's starting point. This ordered set defines a route through the picking zone such that a sequence of picklists $\mathcal{L}$ is completed.

2.3.2. Objective Function. The main factor influencing the total picking time at this operational stage is the picker walking time as most other parts of the picker's time, for example, the time to retrieve items from a shelf, is fixed once a picklist is given. Hence, the GOPP's objective minimizes the total picker walking time, considering the picker's walking time τ_{c_{i-1},c_i} between consecutive vertices in Π

$$Z(\Pi) = \sum_{i \in \{1, \ldots, |\Pi|-1\}} \tau_{c_{i-1},c_i}. \tag{1}$$

2.3.3. Constraints. To properly define all constraints for the GOPP, which differ depending on the type of batching, on the available travel modes, and on the number of drop-off points, we introduce the notion of *active intervals.*

Definition 2 (Active Interval). Given a solution Π, a picker starts a not-yet-processed picklist l at a drop-off point preceding the first storage position visit related to l. Analogously, the picker hands over a completed picklist l at a drop-off point succeeding the last storage position visit related to l. During the time between these two drop-off point visits, we refer to a picklist as active and to the complete time span during which a picklist is active as its active interval.

Then, the constraints for the GOPP are as follows:

i. The cart capacity κ is limited and $|\mathcal{L}_i^{\mathrm{a}}| \leq \kappa$ must hold for any pair π_i.

ii. Once a list starts being processed by a picker, it must be finished before a new picklist can be processed in the respective bin, that is, each picklist has exactly one coherent active interval.

iii. A picklist l cannot be completed before all visits of l are covered in its active interval.

iv. Picklists $\mathcal{L}$ are processed in their given order.

v.s. For static batching, the active intervals of picklists either completely overlap or are completely disjoint.

v.d. For dynamic batching, (iv.s) is relaxed. This allows handing over merely a subset of already completed active orders when stopping at a drop-off point.

vi. Bins can only be handed over at a drop-off point if the cart carrying them is also at the drop-off point, that is, no drop-offs during cartless subtours are allowed.

vii. All picklists must be completed, which is fulfilled when each picklist has exactly one active interval.

viii. If cartless subtours are allowed, only feasible subtours occur, such that the picker's capacity for carrying items is not exceeded, leaving and returning of hand-carried items refer to the same parking position of the cart, and the subtour occurs within one subaisle.

Among all feasible solutions fulfilling these constraints, the GOPP seeks a solution Π^* that minimizes the objective function (1).

3. Methodology

In Online Appendix B, we prove that the GOPP is strongly NP-hard. However, a special case of the GOPP can be solved in polynomial time. Specifically, for a setting with a single picklist, a single travel mode, and a single drop-off point, the rectilinear layout characterizing the GOPP (Figure 1) limits the number of possible ways to traverse a subaisle (see Section 3.1). Accordingly, after summarizing the findings of previous work (Section 3.1), we first develop a skeleton for our algorithmic framework that efficiently solves the GOPP, limited to a single picklist, travel mode, and drop-off point, by using a *layered graph algorithm* (LGA) (Section 3.2). We present a complementary integer linear program (ILP) based skeleton in Online Appendix C. We then derive a unified framework where we incorporate this specific version of the GOPP as a subproblem such that we can incorporate the skeletons to solve all variants of the GOPP based on branching and pruning rules (Section 3.3).

3.1. State of the Art

A reduced version of the GOPP constitutes a special case of the rectilinear TSP that allows us to exploit some properties of this problem. Hence, for the sake of completeness, we summarize state-of-the-art developments in two fields: exact algorithms for picking problems and recent enhancements of exact algorithms for the rectilinear TSP.

Exact algorithms for order picking are still rare (see Section 1.1). To the best of our knowledge, only three algorithms exist, all based on the work of Ratliff and Rosenthal (1983) who presented a first polynomial algorithm for a specified warehouse layout, namely, a warehouse without interspersed cross-aisles. In a nutshell, the NP-hardness of the general TSP is removed because it can be proven that at most two arcs between adjacent vertices exist in a solution. This algorithm scales linearly with the number of subaisles because its complexity is significantly reduced by Proposition 1, which introduces the concept of *transitions* to denote a picker's movement through a subaisle (a, b).

Example 1 (Using Transitions to Model Picker Movements in Subaisles). Given a subaisle (c, c') that starts at c and ends at c', a transition denotes a possible way for a picker to travel through (c, c') in order to collect all items that he or she must pick in this subaisle. Figure 2 shows examples of such transitions for a subaisle (c, c').

Proposition 1 (Adapted from Ratliff and Rosenthal 1983). *Let $G = (\mathcal{V}, \mathcal{A})$ be a graph representation of a rectilinear picking zone. Then, an optimal picker tour, that is, the shortest tour that covers every position with an item that must be picked, can be represented by using only six different transitions for a subaisle (c, c') and three different transitions for a subcross-aisle.*

Figure 3 illustrates the transitions of Proposition 1 for a general representation of (c, c') with two (optional) stops in the subaisle. Besides the transitions already detailed in Figure 2, these transitions contain an empty transition (*I*), as some subaisles that do not contain items that must be picked may not be visited in an optimal solution. Summarizing, the complete set of transitions contains a nontraversed subaisle (*I*) and a complete traversal in one (*II*) or two (*III*) directions. Furthermore, a subaisle may be partially traversed with

Figure 2. Examples of Transitions for a Subaisle

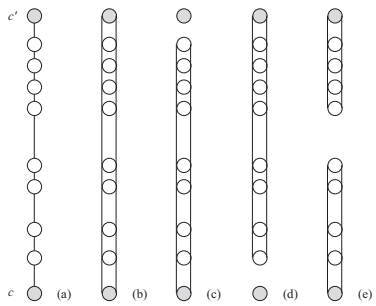
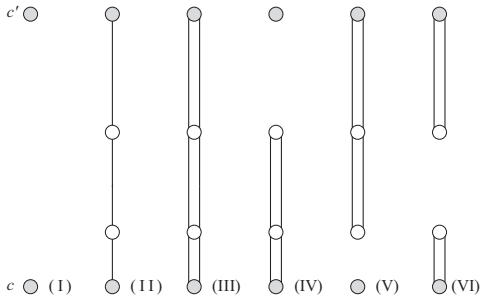


Figure 3. Subaisle Transitions for Optimal Paths in G



slopes from one (*IV*, *V*) or both (*VI*) directions. For transition (*VI*), the split between both slopes is always given by the longest distance between two stops in the same subaisle if more than two stops are present in the subaisle. Although this split criterion may not be unique, it does not affect the solution's global optimality because all split options have the same cost. For cross-aisles in which the picker does not pick up items, only transitions *I*–*III*, from here on referred to as standard transitions, are necessary.

Based on this rationale, Roodbergen and De Koster (2001) and Pansart et al. (2018) have proposed exact algorithms for warehouse layouts with three or more cross-aisles. However, these algorithms are limited in their applicability to the GOPP because multiple picklists, parallel processing of picklists, multiple drop-off points, and multiple travel modes are still not considered.

Cambazard and Catusse (2018) proposed an exact algorithm based on finding an optimal tour for a rectilinear TSP with a set of customers $\mathcal{V}$ by solving a Steiner variant of this problem on the *Hanan graph* $H(\mathcal{V})$ with terminal vertices $\mathcal{V}$. Based on Proposition 2, they decompose the problem and develop a layered graph algorithm that uses a dynamic program to explore $H(\mathcal{V})$ iteratively to find an optimal tour. This algorithm has a computational complexity of $O(nh7^h)$, where n represents the number of customers and h denotes the number of horizontal lines in the Hanan graph $H(\mathcal{V})$.

Definition 3 (Hanan Graph). Let $\mathcal{U}$ be a finite set of vertices in a plane. We first construct a grid, made up of vertical and horizontal lines through each point of $\mathcal{U}$, commonly known as a Hanan grid (Hanan 1966). The resulting intersections define a second set of vertices $\mathcal{S}$, which includes $\mathcal{U}$. The Hanan graph $H(\mathcal{U}) = (\mathcal{S}, \mathcal{E})$ formally defines this grid and consists of the vertex set $\mathcal{S}$ and an edge set $\mathcal{E}$, which contains all edges linking two consecutive vertices of $\mathcal{S}$ in the horizontal or vertical direction.

Proposition 2 (Adapted from Cambazard and Catusse 2018). *Let T^* be an optimal tour in $H(\mathcal{V})$. Then, there exists a planar separator S of $H(\mathcal{V})$ that decomposes T^* into two partial tours T_1^* and T_r^*.*

Definition 4 (Planar Separator). For a Hanan graph $H(\mathcal{U}) = (\mathcal{S}, \mathcal{E})$, we refer to a subset of its vertices $S \subset \mathcal{S}$ as a planar separator if the removal of these vertices partitions the graph into two disjoint subgraphs. We call S a vertical planar separator, if $|S| = \overline{h}$ and $\chi^h(S) = \{1, \ldots, \overline{h}\}$, with $\overline{h}$ denoting the number of different horizontal positions $\{1, \ldots, \overline{h}\}$ in H and $\chi^h(S)$ being a function that denotes the set of horizontal positions for each vertex in S.

3.2. A Bidirectional Layered Graph Algorithm for a Single Picklist

In the following, we develop a bidirectional layered graph algorithm for a specific case of the GOPP with a single picklist and a single drop-off point. We first sketch the general idea of our algorithm in Section 3.2.1, before we detail its main components in Section 3.2.2, and show in Section 3.2.3 how the algorithm's performance can be further enhanced via an improved graph representation.

3.2.1. Algorithm Sketch. Our algorithmic idea is based on four fundamental steps, which we outline in the following, before we formalize our algorithm.

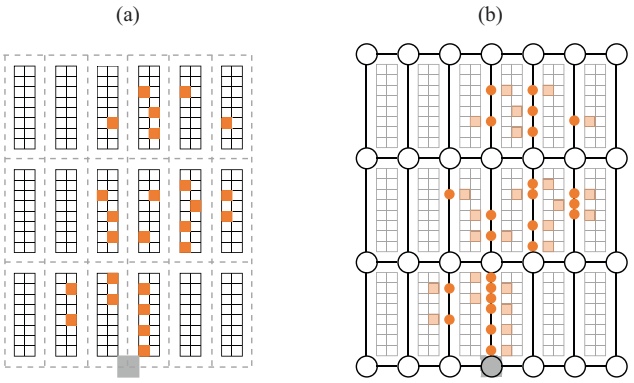
i. We first introduce a compact graph representation for our physical planning problem: Figure 4(a) depicts a picking zone layout with a single drop-off point and one picklist. Here, the drop-off point is the gray rectangle and the shelves that must be visited are highlighted in orange. Figure 4(b) shows its corresponding graph representation. We define a Hanan graph $H(\mathcal{C}) = (\mathcal{C}, \mathcal{E})$ based on the intersections $\mathcal{C}$ of the picking zone and note that its edges $\mathcal{E}$ span all storage positions.

ii. We then note that the transitions defined by Ratliff and Rosenthal (1983) still hold in our setting so that we can use these transitions to describe a picking tour in $H(\mathcal{C})$.

iii. Then, our algorithm's main rationale is to solve a modified Steiner TSP in $H(\mathcal{C})$. This Steiner TSP remains modified from its original definition because instead of a set of terminal vertices, a set of terminal edges $\mathcal{E}^m \subseteq \mathcal{E}$ and the depot vertex must be covered by the tour in its solution. These edges represent subaisles that the picker has to visit to pick all items on his or her list. Here, we use the transitions mentioned above to explore edge traversals and introduce the definition of a *path subgraph* to describe a solution of this Steiner TSP.

iv. In this setting, we use a planar separator to split a path subgraph into *partial path subgraphs*. This allows us to design a problem-specific LGA algorithm to solve the modified rectilinear Steiner TSP and thus the GOPP

Figure 4. (Color online) Example of a Storage Layout



Notes. (a) Physical layout. (b) Graph representation.

with a single picklist, a single travel mode, and a single drop-off point.

Definition 5 (Path Subgraph). A subgraph $T = (\mathcal{V}^T, \mathcal{A}^T)$ of G is a path subgraph if it is connected and its degree $\deg v > 0$, and $\deg v \bmod 2 = 0$ for $v \in \mathcal{V}^T$.

Definition 6 (Partial Path Subgraph). Let $T = (\mathcal{V}^T, \mathcal{E}^T)$ be a subgraph of $L = (\mathcal{V}^L, \mathcal{E}^L)$ that is a subgraph of $G = (\mathcal{V}^G, \mathcal{E}^G)$. Then, T is an L partial path subgraph of G if there exists at least one subgraph $F = (\mathcal{V}^F, \mathcal{E}^F)$ with $\mathcal{E}^F \subset \mathcal{E}^G \setminus \mathcal{E}^L$ so that T and F form a path subgraph in G.

Compared with a standard LGA implementation as in Cambazard and Catusse (2018), we make the following extensions to derive an algorithm suitable to solve the GOPP with a single picklist and depot: as indicated above, we introduce $\mathcal{E}^{\mathrm{m}} \subseteq \mathcal{E}$ as the subset of edges that must be visited, from here on referred to as mandatory edges, to ensure that we visit all pick positions where items must be picked. Moreover, we develop a bidirectional algorithm that splits $H(\mathcal{C})$ into two subgraphs, explores these subgraphs individually, and then merges the partial solutions to a complete solution.

Although the worst-case complexity of bidirectional algorithms often remains the same as that of their monodirectional counterparts or even gets worse because of an additional merge procedure, empirical evidence for various problems showed superior performance of bidirectional approaches, for example, in branch-and-price algorithms. Assuming a computationally efficient merge procedure, a bidirectional algorithm does not perform worse than a monodirectional algorithm. It may even be better because exploring the solution space from different directions in two separate searches may reduce the number of states that must be explored, partially by detecting additional dominance relations. As our merge procedure remains computationally efficient (see Online Appendix E), it seems promising to develop a bidirectional search. Although this algorithm was developed for the picking problem, it also appears to be the first that can readily be adapted to the rectilinear Steiner TSP.

3.2.2. Algorithmic Components. To formalize our algorithm, we first label vertical positions v and horizontal positions h in $H(\mathcal{C}) = (\mathcal{C}, \mathcal{E})$. Here, $h \in \{1, \dots, \overline{h}\}$ refers to the h^{th} horizontal line, that is, in our specific setting, to the h^{th} cross-aisle. We label horizontal lines from the lower left corner ($h = 1$) to the upper left corner ($h = \overline{h}$). Analogously, $v \in \{1, \dots, \overline{v}\}$ refers to the v^{th} vertical line, that is, in our specific setting, to the v^{th} aisle. We label vertical lines from the lower left corner ($v = 1$) to the lower right corner ($v = \overline{v}$). We then uniquely identify a vertex $c \in \mathcal{C}$ by its location on the h^{th} horizontal and the v^{th} vertical line of $H(\mathcal{C}) = (\mathcal{C}, \mathcal{E})$. Let $\chi^{\mathrm{h}}(c)$ denote the horizontal position of a vertex with $\chi^{\mathrm{h}}(c) \in \{1, \dots, \overline{h}\}$ and let $\chi^{\mathrm{v}}(c)$ denote the vertical position of a vertex c with $\chi^{\mathrm{v}}(c) \in \{1, \dots, \overline{v}\}$. We use both functions for single vertices as defined above but also for sets of vertices, then returning a set of positions.

With this notation, we define a *central planar separator* $\hat{S}$, which allows to define two subgraphs B and U of $H(\mathcal{C})$ as shown in Figure 5. Here, B consists of all vertices in $\hat{S}$ and to the left of $\hat{S}$ and its corresponding edges, whereas U consists of all vertices in $\hat{S}$ and to the right of $\hat{S}$ and its corresponding edges. In each of these subgraphs, we define additional subgraphs B_{hv}/U_{hv}, where the edges of B_{hv} are induced by all vertices $c \in B$ for which $(\chi^{\mathrm{v}}(c) < v) \vee (\chi^{\mathrm{v}}(c) = v \wedge \chi^{\mathrm{h}}(c) \leq h)$ holds, and the edges of U_{hv} are induced by all vertices $c \in U$ for which $(\chi^{\mathrm{v}}(c) > v) \vee (\chi^{\mathrm{v}}(c) = v \wedge \chi^{\mathrm{h}}(c) \geq h)$ holds. Then, let R_{hv}/L_{hv} be the planar separators that defines B_{hv}/U_{hv} in B/U. Figure 6 shows an example of a subgraph B_{34} with its separator R_{34} in B and a subgraph U_{46} with its separator L_{46} in U.

Definition 7 (Central Planar Separator). We refer to a planar separator S of a Hanan graph $H(\mathcal{U}) = (\mathcal{S}, \mathcal{E})$ as central, if

Figure 5. (Color online) Example of $\hat{S}$ and Its Resulting Subgraphs

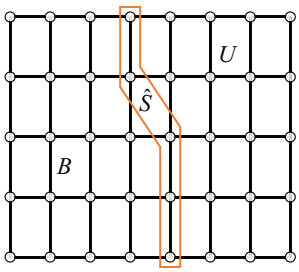
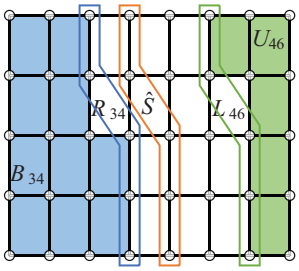


Figure 6. (Color online) Example of Subgraphs B_{34} and U_{46}



1. S is a vertical planar separator according to Definition 4.

2. For all $c_i, c_j \in S$, it holds that $\chi^{\mathrm{v}}(c_j) \leq \chi^{\mathrm{v}}(c_i)$ if $\chi^{\mathrm{h}}(c_j) > \chi^{\mathrm{h}}(c_i)$ and $|\chi^{\mathrm{v}}(S)| = 2$.

3. $\|\mathcal{E}'| - |\mathcal{E}''\|$ is minimal for the resulting subgraphs $H' = (\mathcal{S}', \mathcal{E}')$ and $H'' = (\mathcal{S}'', \mathcal{E}'')$, with H' being induced by all vertices in or to the left of S and H'' being induced by all vertices in or to the right of S.

Recall that Proposition 2 suggests that we can explore B and U independently to find a cost-minimal path subgraph in $H(\mathcal{C})$, that is, a shortest tour for the Steiner TSP. Then, the main rationale of our algorithm is to propagate partial path subgraphs in B and U in order to complete these to a cost-minimal path subgraph. To propagate these partial path subgraphs, we use a transition set $\mathcal{T}_e$ which denotes all aforementioned possible transitions $t \in \mathcal{T}_e$ for each edge $e = (c, c') \in \mathcal{E}$.

We develop a bidirectional LGA that iteratively explores subgraphs B_{hv}, U_{hv} in B, U to identify partial path subgraphs. As each subgraph B_{hv}/U_{hv} can contain multiple partial path subgraphs, we identify each partial path subgraph by a state α. Each state $\alpha = \{\mathbf{r}, \mathbf{u}\}$ consists of a vector $\mathbf{r} = (r_1, \ldots, r_{\bar{h}})$, which denotes the degree parity of each vertex in R_{hv}/L_{hv}, and of a vector $\mathbf{u} = (u_1, \ldots, u_{\bar{h}})$, which denotes the *connected component* to which each vertex in R_{hv}/L_{hv} belongs. Here, both the i^{th} entry r_i of $\mathbf{r}$ and the i^{th} entry u_i of $\mathbf{u}$ state the degree parity, respectively, the connected component label, of the vertex that lies on the i^{th} horizontal line in R_{hv}/L_{hv}.

Definition 8 (Connected Component). A partial path subgraph may contain partial paths that are not necessarily connected with each other but may still yield a feasible solution if they are merged with a second partial path subgraph that preserves the connectedness of the overall path.

To trace the number of different partial paths, we use a connected component label u_i given by an integer number. Here, u_i denotes the connected component label of the vertex that lies on the i^{th} horizontal line of $H(\mathcal{C})$ in the planar separator. We initialize the label of the first explored vertex with one and increase the label every time a transition disconnects c' from c when propagating (c, c'). When a transition reconnects two vertices, both gain the lower connected component label.

Once $L_{hv} = R_{hv} = \hat{S}$, it holds that $B_{hv} = B$ and $U_{hv} = U$ and B and U collectively exploit $H(\mathcal{C})$ exhaustively (Figure 7). We can then identify the optimal path by merging feasible paths from B_{hv}/U_{hv}. To specify our general algorithmic concept, we now define transitions to propagate partial path subgraphs, a feasibility check to either discard infeasible states or to merge partial path subgraphs, and a dominance criterion to discard dominated states.

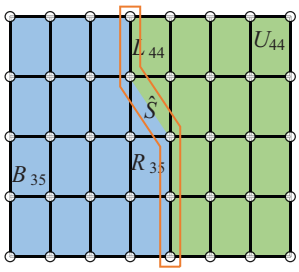
3.2.2.1. Transitions.

We use the transitions proposed by Ratliff and Rosenthal (1983) as vertical transitions (cf. Figure 3). Doing so straightforwardly yields a transition set that is larger than needed. To derive a minimum transition set and speed up computational times, we separate all edges of set $\mathcal{E}$ of $H(\mathcal{C})$ into a set of mandatory edges $\mathcal{E}^{\mathrm{m}}$ and a set of optional edges $\mathcal{E}^{\mathrm{o}}$. Mandatory edges comprise edges that represent a mandatory subaisle, whereas optional edges represent nonmandatory subaisles or cross-aisles such that $\mathcal{E} = \mathcal{E}^{\mathrm{m}} \cup \mathcal{E}^{\mathrm{o}}$. Because each edge in $\mathcal{E}^{\mathrm{m}}$ must be visited, transitions *II*–*VI* are sufficient to propagate mandatory edges. Nonmandatory edges will only be used as connecting edges such that we need only transitions *I*–*III* to propagate these.

3.2.2.2. Feasibility Check.

We use the information stored in the state $\alpha = \{\mathbf{r}, \mathbf{u}\}$ to evaluate whether a transition leads to an infeasible state or not. Our feasibility check then bases on the degree $r_i \in \mathbf{r}$ and on the connected component label $u_i \in \mathbf{u}$ of each vertex in R_{hv}/L_{hv}. The degree of each vertex $r_i \in \{\mathrm{E}, \mathrm{O}\}$ can be even (E) or odd (O). A partial path subgraph in B_{hv}/U_{hv} may contain multiple partial paths that are not necessarily connected with each other (cf. Definition 6). To trace these, u_i denotes an integer label of the connected component

Figure 7. (Color online) Fully Explored Graph with $\hat{S} = L_{44} = R_{35}$



of the partial path ending on the i^{th} horizontal line in R_{hv}/L_{hv}.

Given this information, we can check the feasibility of a transition as follows. A transition leads to an infeasible state if any of the following statements holds:

i. After applying a transition from vertex c to vertex c', c remains with an odd degree but is no longer in R_{hv}/L_{hv}.

ii. After applying a transition, the drop-off point vertex is not connected to the path.

iii. After applying a horizontal transition, a connected component is reduced without a merge.

To check the feasibility during the merge operation, a merge is feasible only if

i. No vertex in $\hat{S}$ remains with an odd degree.

ii. Only a single connected component label remains.

3.2.2.3. Dominance Criterion.

We use the definition of *equivalence* between partial path subgraphs to develop a dominance criterion. To keep the number of propagated partial path subgraphs as small as possible, we withdraw each partial path subgraph that is equivalent to an already existing partial path subgraph and does not improve its cost. This criterion is based on the rationale that for two *equivalent partial path subgraphs* T and T' in B_{hv}, it is sufficient to keep only the one with the lower cost to derive an optimal path subgraph. We establish this equivalence in Proposition 3.

Definition 9 (Equivalent Partial Path Subgraphs). We consider a Hanan graph $H(\mathcal{C})$ and its subgraphs B and U that collectively cover $H(\mathcal{C})$ exactly. Let T and T' be two distinct partial path subgraphs in B. We call T and T' equivalent if any partial path subgraph F in U that completes T to a path subgraph in $H(\mathcal{C})$ also completes T' to a path subgraph in $H(\mathcal{C})$.

Proposition 3. *We consider a Hanan graph $H(\mathcal{C})$ and its subgraphs B and U that collectively cover $H(\mathcal{C})$ exactly. Let T and T' be two distinct partial path subgraphs in B. Then, T and T' are equivalent if their states α and α' are equal.*

Using the described transitions, feasibility check, and dominance check, we propagate partial path subgraphs until B_{hv} and U_{hv} collectively exploit $H(\mathcal{C})$ when $B_{hv} = B$ and $U_{hv} = U$. For the algorithmic details, we refer the interested reader to Online Appendix D.

3.2.3. Improved Graph Representation.

With the modifications described above, we obtain an LGA to solve a picker routing problem for a single picklist, a single drop-off point, and an arbitrary number of aisles and cross-aisles. We now derive an improved graph representation that helps reduce the number of developed labels to enhance the algorithm's computational performance.

The number of labels increases with the number of edges that must be traversed. Hence, we aim to derive a maximal sparse graph that preserves optimality but contains as few edges as possible. We introduce the set $\mathcal{C}^{\text{m}}$ as the set of all vertices that are either adjacent to an edge in $\mathcal{E}^{\text{m}}$ or represent a drop-off point. Then, a graph that preserves the realization of a shortest path between any $c, c' \in \mathcal{C}^{\text{m}}$ preserves optimality. In the following, we provide a reduction technique that improves the sparsity of $H(\mathcal{C})$ while preserving optimality.

Focusing on a planar separator S, we say that this separator is *decreasing* if $\forall c, c' \in S, \chi^{\text{h}}(c') > \chi^{\text{h}}(c) \Rightarrow \chi^{\text{v}}(c') \leq \chi^{\text{v}}(c)$ and *increasing* if $\forall c, c' \in S, \chi^{\text{h}}(c') > \chi^{\text{h}}(c) \Rightarrow \chi^{\text{v}}(c') \geq \chi^{\text{v}}(c)$. With this notation, we state Proposition 4, which allows to construct a reduced Hanan graph.

Proposition 4. *Let $H' = (\mathcal{C}', \mathcal{E}')$ be a reduced Hanan graph with $\mathcal{E}' \subset \mathcal{E}$ being its edge set, induced by the vertex set $\mathcal{C}' \subset \mathcal{C}$. Then, H' is sparser than $H(\mathcal{C})$, that is, $|\mathcal{C}'| < |\mathcal{C}|$, $|\mathcal{E}'| < |\mathcal{E}|$ and preserves optimality if we construct $\mathcal{C}'$ as follows.*

1. *We consider two planar separators S^{l} and S^{r}, for which the following conditions hold:*

   a. $|S^{\text{l}}| = |S^{\text{r}}| = h$ *and* $\chi^{\text{h}}(S^{\text{l}}) = \chi^{\text{h}}(S^{\text{r}}) = \{1, \dots, \overline{h}\}$,

   b. *each planar separator $S^{\text{l}}, S^{\text{r}}$ can be split into two partial planar separators with $S^{\text{l}} = S^{\text{l}}_1 \cup S^{\text{l}}_2$ and $S^{\text{r}} = S^{\text{r}}_1 \cup S^{\text{r}}_2$ such that*

      i. $\forall c \in S^{\text{l}}_1, c' \in S^{\text{l}}_2 : \chi^{\text{h}}(c) \leq \chi^{\text{h}}(c')$,

      ii. $\forall c \in S^{\text{r}}_1, c' \in S^{\text{r}}_2 : \chi^{\text{h}}(c) \leq \chi^{\text{h}}(c')$,

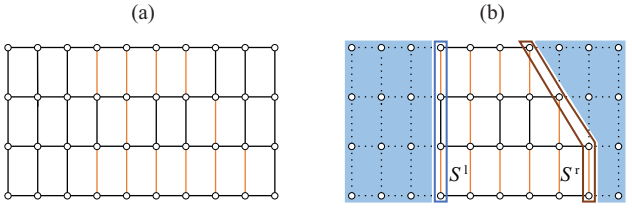
      iii. *S^{l}_1 and S^{r}_2 are decreasing, and S^{l}_2 and S^{r}_1 are increasing.*

2. *We can then construct $\mathcal{C}'$ in $O(\overline{h}\overline{v})$ such that $\mathcal{C}^{\text{m}} \subseteq \mathcal{C}'$ and*

$$\chi^{\text{h}}(c') = \chi^{\text{h}}(c) = \chi^{\text{h}}(c'') \Rightarrow \chi^{\text{v}}(c') \leq \chi^{\text{v}}(c) \leq \chi^{\text{v}}(c'') \quad \forall c \in \mathcal{C}', c' \in S^{\text{l}}, c'' \in S^{\text{r}}.$$

Figure 8 shows an example of such a reduced Hanan graph.

Figure 8. (Color online) Example of a Feasible Reduced Hanan Graph



Notes. (a) Example instance. (b) Feasible reduced Hanan graph.

3.3. Extensions for the General GOPP

We now present a general framework that allows to integrate the LGA algorithm of Section 3.2 to solve any variant of the GOPP. The main rationale of this framework leverages a branch-and-prune algorithm to make decisions on (i) which subsequent picklists should be processed in parallel and on (ii) the drop-off points at which finished lists should be dropped. To calculate the overall picker walking time for a specific configuration of parallel-processed picklists and drop-off points, we need to solve extended variants of the basic problem studied in Section 3.2. Here, our branch-and-prune algorithm makes use of extended variants of our LGA to solve these subproblems. In the following, we first detail the general branching and pruning framework that embeds our LGA to solve the resulting subproblems in Section 3.3.1. We then discuss characteristics of and extensions that are necessary to integrate *cartless subtours*, a *dynamic batching policy*, and *multiple drop-off points* into our LGA in Section 3.3.2, and derive lower bounds in order to speed-up the branch-and-prune algorithm in Section 3.3.3. We note that we could also use our ILP within the branch-and-prune algorithm and refer to Online Appendix C for further elaboration.

3.3.1. Branch-and-Prune Algorithm. As a prerequisite to formalize our branch-and-prune algorithm, we first decompose the solution of the GOPP. To this end, we refer to a solution $\sigma = (\vartheta_1, \dots, \vartheta_n)$ as an n-tuple of subproblem solutions ϑ_i. Each solution is associated with a cost $C(\sigma) = \sum C(\vartheta_i)$. A subproblem solution represents a single picking tour between two drop-off point visits and is a quadruple $\vartheta_i = (\mathcal{L}_i^{\mathrm{a}}, \mathbf{T}_i, \omega_i^-, \omega_i^+)$ consisting of a set of active picklist labels $\mathcal{L}_i^{\mathrm{a}} \subseteq \mathcal{L}$ which depends on the general set of order list labels $\mathcal{L}$ to be processed, a vector of transitions $\mathbf{T}_i$ that comprises an explicit transition $t_{ij} \in \mathcal{T}_{ij}$ for each edge, a starting drop-off point ω_i^-, and a final drop-off point ω_i^+.

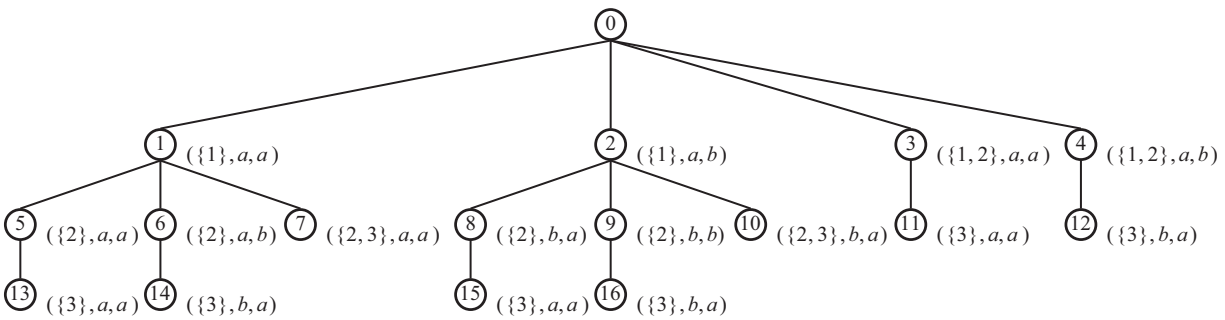
To solve the GOPP using this decomposition, one needs to decide on (i) $\mathcal{L}_i^{\mathrm{a}*}$, that is, which picklists are processed in which subproblem, (ii) which drop-off points $\omega_i^{+*}, \omega_i^{-*}$ should be used in each subproblem, and on (iii) $\mathbf{T}_i^*$, that is, which transitions form the cost optimal picking tour. To make decision (iii) and obtain $\mathbf{T}_i^*$, we leverage an extended variant of our LGA, which we detail in Section 3.3.2. We now develop a branch-and-prune algorithm that allows to make decisions (i) and (ii), using our extended LGA as an oracle to determine the minimum cost of each subproblem.

We represent our branch-and-bound tree as a graph-theoretic tree, with $\mathcal{N}$ being the set of node labels of the complete tree, 0 being its root node, and $\mathcal{Z}_n \subset \mathcal{N}$ denoting the child nodes of node n. Each node n in this tree represents a subproblem θ_n, identified by a unique triple $\theta_n = (\mathcal{L}_n^{\mathrm{a}}, \omega_n^+, \omega_n^-)$, which may be part of the optimal solution that is given by the cost-minimum path from the tree's root node to a leaf.

Example 2 (Branch-and-Bound Tree). We consider a set of picklists $\mathcal{L} = \{1,2,3\}$, a picking zone layout with two drop-off points $a, b \in \mathcal{D}$, and a cart capacity of $\kappa = 2$. Further, the picker is supposed to start his or her first tour and to end his or her last tour at a. Figure 9 shows the corresponding fully enumerated branch-and-bound tree, which contains subproblems with four different active picklist configurations $\mathcal{L}_i^{\mathrm{a}} \in \{\{1\}, \{2\}, \{3\}, \{1,2\}, \{2,3\}\}$, and depot configurations $(\omega_i^-, \omega_i^+) \in \{(a,b), (b,a), (a,a), (b,b)\}$. As one can see, the leaving drop-off point in a child subproblem always remains the entering drop-off point in its direct parent subproblem. This ensures that the subproblems capture the total picker walking distance. Moreover, a path's depth from the root node to a leaf may vary, depending on the configuration of active order lists in each subproblem, because a path up to a leaf must always contain all picklists to represent a feasible solution.

We introduce the following additional notation to detail the algorithm that allows us to explore such a tree efficiently. We associate each node n with a subproblem sequence $\Theta_n = (\theta_i, \dots, \theta_n)$, defined by the path from the root node zero to node n. For example, referring to the example given in Figure 9, the subproblem sequence defined by node $n = 6$ reads $\Theta_6 = (\theta_1, \theta_6)$, whereas the subproblem sequence for node $n = 16$ is $\Theta_{16} = (\theta_2, \theta_9,$

Figure 9. Branch-and-Bound Tree Representation for $|\mathcal{L}| = 3$, $\kappa = 2$, and $|\mathcal{D}| = 2$



θ_{16}). Note that the root node is never part of such a sequence as it is not associated with a subproblem, because the first subproblem is not unique due to the varying active picklist configurations.

With ϑ_i being the solution to subproblem θ_i, we can then define the cost C_n of a node n as the sum of the subproblem costs of its corresponding subproblem sequence and note that a subproblem sequence Θ whose subproblems cover all picklists which must be processed constitutes a complete solution σ. We use σ^* to denote the optimal solution and C^* to denote its cost. Further, let $\psi(n)$ denote a function that calculates a lower bound on the cost of a solution that includes the subproblem sequence of n. We refer to Section 3.3.3 for details on the lower bound calculation and note that for leaves of the tree, $\psi(n) = C_n$. To explore the tree, we use sets $\mathcal{N}^a$ and $\mathcal{N}^s$ to keep track of nodes that have not been solved ($\mathcal{N}^a$) and nodes that have already been solved ($\mathcal{N}^s$). Here, $\mathcal{N}^a$ remains an ordered set that can be seen as a priority queue which contains nodes hierarchically ordered by (i) the number of processed order lists, starting with the maximum, and (ii) nodes with an equal number of processed order lists in increasing order with respect to their lower bound cost. Whenever we insert nodes into $\mathcal{N}^a$, we preserve this order.

3.3.1.1. Subproblem Sequence Dominance.

Clearly, this branch-and-bound tree grows significantly for larger problem settings depending on the number of picklists and drop-off points. To allow for a tractable exploration of such a tree, we introduce the following dominance relation. We consider two subproblem sequences Θ_n and $\Theta_{n'}$. We then say that Θ_n dominates $\Theta_{n'}$ ($\Theta_n \succ \Theta_{n'}$) if (i) the picklists processed in $\Theta_{n'}$ have also been processed in Θ_n and (ii) $C_n + C(\omega_n^+, \omega_{n'}^+) \leq C_{n'}$, with $C(\omega_n^+, \omega_{n'}^+)$ being the cost to move from ω_n^+ to $\omega_{n'}^+$. This bound allows to discard a majority of nodes of the branch-and-bound tree early and thus allows to efficiently solve large-size instances.

3.3.1.2. Pseudo-Code.

Figure 10 details the pseudo-code of our algorithm. We first initialize $\mathcal{N}^a$ with the children of the root node $\mathcal{Z}_0$ and set C^* to infinity (l.1). We then explore nodes in $\mathcal{N}^a$ in a branch-and-bound fashion. We explore the tree with a depth-first strategy, defined by the order of $\mathcal{N}^a$; in case of a tie, we chose the node with the smaller index (l.3). We then check whether the node is dominated (l.4) or it's lower bound exceeds the so far best-found solution cost (l.5). If any of these checks is true, we discard (prune) the node and continue with the next node in $\mathcal{N}^a$. If none of these checks is true, we either update the optimal solution if the current node is a leaf (l.6–8) or add n' to $\mathcal{N}^s$ (l.10) and add its children to $\mathcal{N}^a$ (l.11). We note

Figure 10. Pseudo-Code for Our Branch-and-Prune Algorithm

```
1:  N^a ← Z_0, C* ← ∞
2:  while N^a ≠ ∅ do
3:      n' ← getFirstNode(N^a)
4:      if isNotDominated(n') then
5:          if ψ(n') < C_{n*} then
6:              if isLeafNode(n') then
7:                  C* ← C_{n'}
8:                  σ* ← Θ_{n'}
9:              else
10:                 N^s ← N^s ∪ {n'}
11:                 extend(N^a, Z_{n'})
12: return σ*
```

that the smallest lower bound calculated by $\psi(n)$ is usually not tight, that is, we do not achieve an exact matching between the smallest lower bound and the global upper bound. However, the lower bound often allows to discard unfavorable unexplored subproblems early.

3.3.2. Subproblem Extensions.

We recall from Section 3.3.1 that the cost of a subproblem solution results from the cost of its transitions. Accordingly, we can determine the cost of a subproblem by computing a cost-optimal transition vector using the LGA presented in Section 3.2, accounting for the following modifications in settings with cartless subtours, multiple drop-off points, and dynamic batching. We note that all of the detailed modifications are additive, for example, to solve a subproblem with cartless subtours, multiple drop-off points, and dynamic batching, we apply all modifications outlined below but no further interdependencies between those modifications apply.

3.3.2.1. Cartless Subtours.

Allowing for cartless subtours results in varying travel speeds in subaisles, which affect the cost of the transitions. We prove in Proposition 5 that the transitions discussed in Section 3.2 still suffice to calculate an optimal solution. Thus, our algorithm can be applied to problems with cartless subtours without changes.

Proposition 5. *If a picker can switch the travel mode from picking SKUs accompanied by his or her cart to picking a limited number of items of the same or different SKUs on foot in line with the definition of a cartless subtour, the transition set $\mathcal{T}$ is sufficient to derive an optimal solution.*

Therefore, no changes to our algorithm are necessary to handle cartless subtours. We only add a routine that considers potential cartless subtours and the corresponding travel time reductions when calculating the cost of all transitions during preprocessing.

3.3.2.2. Multiple Drop-off Points. We recall that the decision on which drop-off points are used is made in the branch-and-prune phase of our algorithm. This reduces the necessary modifications in our LGA to account for settings where the starting drop-off point and the final drop-off point are different, that is, $\omega_i^+ \neq \omega_i^-$. Such a setting leads to optimal subproblem solutions with tours that contain odd-degree vertices, which are prohibited by the feasibility check of our basic LGA, as detailed in Section 3.2. We prove that these odd degrees are always limited to the drop-off point vertices ω_i^+ and ω_i^-, that is, the start and end of a subproblem's optimal tour.

Proposition 6. *The optimal path subgraph T^* of ϑ_i with $\omega_i^+ \neq \omega_i^-$ comprises exactly two vertices with an odd degree that relate to ω_i^+ and ω_i^-.*

With Proposition 6, we extend the feasibility check of our LGA to a subproblem with $\omega_i^+ \neq \omega_i^-$ by enforcing an odd instead of an even degree for ω_i^+ and ω_i^-.

3.3.2.3. Dynamic Batching. Under a dynamic batching policy, a picker can pick multiple lists $l \in \mathcal{L}$ at once (as long as the cart's bin capacity κ is not exceeded) and may return to the drop-off point once at least one but not necessarily all picklists are finished to empty one or several bins. As explained in Section 3.3.1, we model such possibilities with subproblem variations: each subproblem comprises the possible transitions to form a picker tour between two drop-off point visits, where the transitions result from the order lists that are active in the respective subproblem.

Specifically, this implies that subproblems exist where the number of active order lists is lower than the bin capacity to reflect cases where a subset of bins is returned to a depot early, although collecting a larger number of picklists in parallel. In these cases, we calculate the respective subproblem's transitions based on the active picklists but consider items of subsequent picklists that are processed in parallel as picked if the resulting transitions cover them. This implicit picking of items from inactive lists reduces the number of items to be picked when their picklist becomes active in a subsequent subproblem, without delaying the drop-off of active picklists. Here, one may achieve even shorter overall picking times by picking more items early from subsequent lists, at the price of delayed drop-offs of active lists. Accounting for this characteristic would require a modified algorithm and remains an interesting future research direction. Here, the decision on which picklists are active in parallel influences the number of consecutive subproblems in a subproblem sequence Θ (see Example 2). We recall that order lists must be processed in chronological order (cf. Section 2.3) and note that this limits the combinations of active picklists.

3.3.3. Lower Bounds for a Single Subproblem. In the following, we detail the function $\psi(n)$ that we use in Section 3.3.1 to efficiently prune the branch-and-bound tree. Specifically, $\psi(n)$ calculates a lower bound on the cost of a solution that includes the subproblem sequence of n by extending C_n with the maximum of two lower bounds Υ_1 and Υ_2. Here, Υ_1 and Υ_2 are lower bounds for the remaining unexplored subproblems' cost that can be added to the subproblem sequence to get a complete solution. For each unexplored subproblem, we calculate Υ_1 and Υ_2 as follows.

To calculate Υ_1, we consider our original graph representation $G = (\mathcal{V}, \mathcal{A})$. We derive a minimal spanning tree on a reduced graph $G^c = (\mathcal{V}^c, \mathcal{E}^c)$, where $\mathcal{V}^c$ contains the drop-off vertices and every pick-up position of the subproblem, $\mathcal{E}^c = \{\{i,j\} \mid i \in \mathcal{V}^c, j \in \mathcal{V}^c\}$, and G^c is weighted with the walking time τ_{ij} that results from the Manhattan distance of each arc. Then, a minimal spanning tree is given by $P^c \subseteq \mathcal{E}^c$ with a minimal total weight that yields Υ_1 such that $\Upsilon_1 = \sum_{(i,j)\in P^c} \tau_{ij}$.

To calculate Υ_2, we consider the reduced Hanan graph $H' = (\mathcal{E}', \mathcal{C}')$. We derive a lower bound by summing up (i) the cost-minimum transition for each mandatory edge, that is, $\zeta_{ij}^* = \min_{t\in\mathcal{T}_{ij}}\{\zeta_{ijt}\}$, and (ii) costs ζ_{III} for a double traversal of cross-aisles between each pair of aisles with n being the number of aisles remaining in H' such that $\Upsilon_2 = \sum_{(i,j)\in\mathcal{E}^m} \zeta_{ij}^* + \zeta_{III}(n-1)$. For cartless subtours, we treat all transitions of type *IV*, *V*, and *VI* as cartless to calculate Υ_2.

If the leaving and returning depots are not equal, we adapt the calculation of these bounds accordingly, that is, in this case, the spanning tree for Υ_1 comprises both drop-off vertices, and the minimum cost transition calculation for Υ_2 considers a single transition for both depots.

4. Computational Study

The scope of our computational study is twofold. First, we benchmark the basic component of our algorithm against the current state of the art and we also analyze the performance of our algorithmic framework for real-world environments in Section 4.1. Second, we study various warehouse layouts and picking policies based on three case studies from a managerial perspective in Section 4.2.

We conducted all experiments on a standard desktop computer with an Intel Core I7 3.6 GHz and 16 GB RAM. Our algorithm was implemented as a single thread code in C++. We solved the ILP using Gurobi 8.1.

4.1. Computational Performance

Currently, no algorithm can solve all problem variants considered in this study. The most generic algorithm proposed in parallel to this work by Pansart et al. (2018) focuses on a picker routing problem for a multi-block warehouse limited to a single picklist, a single drop-off point, a static batching, and lacks cartless subtours. We compare the performance of both our monodirectional and our bidirectional LGA and our ILP (see Online Appendix C) to the algorithm of Pansart et al. (2018). Because these authors also use dynamic programming, we reimplemented their algorithm in our framework to allow for a fair comparison on the same desktop computer. The benchmark from Theys et al. (2010), which we detail in Online Appendix G.1, forms the basis for our comparison.

Table 2 compares the performance of our monodirectional (M-LGA) and bidirectional (B-LGA) algorithms, the algorithm of Pansart et al. (2018) (PCC), and our ILP in terms of (i) the number of solved instances and (ii) the computational times. As can be seen, our LGAs perform much better than the algorithm of Pansart et al. (2018) in terms of both computational times and scalability on large-size instances. In particular, for large-size instances, our LGAs show much better computational times and solve more instances. Comparing our monodirectional algorithm to our bidirectional algorithm, one can see that the bidirectional algorithm shows the best computational times and solves more instances with 11 cross-aisles then the monodirectional algorithm. Note that the computational times for the monodirectional algorithm on instances with 11 cross-aisles and five aisles only appear to be lower because less instances are solved. Accordingly, the comparison is not complete as only solved instances are counted within the average computational times. Our ILP shows longer computational times than all problem-specific algorithms but manages to solve a large subset of instances. For the instances with three or six cross-aisles reported as not solved, it still finds an optimal solution but struggles to close the optimality gap within the time limit of an hour. Remarkably, the ILP solves the most instances with 11 cross-aisles out of all algorithms. Although our bidirectional LGA solves 64 instances with 11 cross-aisles, the ILP solves 110 instances with 11 cross-aisles. For most other instances with 11 cross-aisles, the ILP finds a solution but cannot prove its optimality within the given time limit. Only 35 instances remain completely unsolved.

The different performance of these solution approaches can be attributed to specific algorithmic components. Our LGA-based algorithms outperform that of Pansart et al. (2018) for three reasons: first, we use an elaborate online graph reduction that significantly reduces the number of states that must be explored. Notably, the time of this reduction routine is included within the computational times of Table 2. Second, we use improved transition sets that allow for a further reduction in the number of states that must be explored. Third, our feasibility check and equivalence criterion allow us to discard dominated or infeasible states early. In total, the largest share of improvement is due to the graph reduction. Our bidirectional LGA outperforms its monodirectional counterpart as it can discard states earlier by exploring the subgraphs in two directions. Although the computational complexity of our LGAs depends on the number of cross-aisles of an instance, the computational complexity of the ILP depends on the total number of arcs. Accordingly, it succeeds in solving a large number of instances with 11 cross-aisles and a small number of aisles but struggles on closing optimality gaps for instances with many aisles, independent of the number of cross-aisles.

In real-world settings, the computational performance of our algorithm can be better or worse than for

Table 2. Number of Solved Instances and Computational Times in Seconds on the Theys et al. (2010) Benchmarks

		Volume based									Random								
Number of cross-aisles		3			6			11			3			6			11		
Number of aisles		5	15	60	5	15	60	5	15	60	5	15	60	5	15	60	5	15	60
Instances solved	B-LGA	60	60	60	60	60	60	33	15	2	60	60	60	60	60	60	14	2	—
	M-LGA	60	60	60	60	60	60	27	13	2	60	60	60	60	60	60	9	2	—
	PCC	60	60	60	60	60	60	—	—	—	60	60	60	60	60	60	—	—	—
	ILP	60	60	45	60	38	8	26	10	—	60	60	45	60	49	9	44	30	—
Time	B-LGA	0.004	0.01	0.03	0.16	6.87	45.8	337	555	1,320	0.00	0.01	0.04	0.28	10.1	60.1	340	484	—
	M-LGA	0.004	0.01	0.04	0.78	9.42	48.6	153	566	1,803	0.00	0.01	0.04	1.39	13.8	63.1	334	1476	—
	PCC	0.006	0.03	0.09	12.4	193	1062	—	—	—	0.01	0.03	0.09	12.9	193	1,062	—	—	—
	ILP	0.05	10.4	176	12.2	273	1.17	315	778	—	0.05	3.55	174	2.24	125	1.01	98.9	639	—

Note. The table shows the average computational times of the solved instances on the Theys et al. (2010) benchmark for the algorithm of Pansart et al. (2018) (PCC), our monodirectional LGA (M-LGA), our bidirectional LGA (B-LGA), and our ILP.

Table 3. Computational Results for Benchmark Instance Sets with 50 Aisles

Number of cross-aisles	3	4	5	6	7	8
Average computational time (s)	0.02	0.13	1.54	25.6	322	—
Average speed-up factor	1.50	1.66	3.16	6.08	16.4	—
Maximum value speed-up factor	3.07	4.77	20.4	64.5	71.0	—
Third quartile speed-up factor	1.46	1.61	2.75	3.87	16.9	—
Median speed-up factor	1.28	1.43	2.09	2.52	6.49	—
First quartile speed-up factor	1.20	1.30	1.67	1.92	3.14	—
Minimum value speed-up factor	1.06	1.09	1.19	1.20	1.30	—

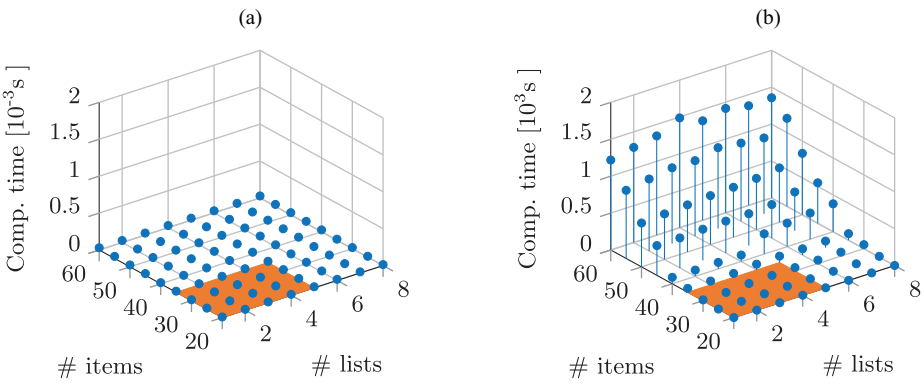
Note. The table shows the average computational time and the statistical characteristics of the ratio between the computational time without and with preprocessing for benchmark sets of 20 instances for different numbers of cross-aisles.

the Theys et al. (2010) benchmark because of a multitude of complexity drivers. Hence, we use a second benchmark, detailed in Online Appendix G.1, to analyze the computational performance of our algorithm with respect to (i) the number of cross-aisles, (ii) the number of processed picklists and the number of items on each list, and (iii) the picking zone layout. Further, we show the impact of our preprocessing technique.

Table 3 shows computational results for a set of benchmark instances with a varying number of cross-aisles. We created 20 instances with a single depot and 50 aisles for each cross-aisle setting, generating pick positions as described in Online Appendix G.1. We report the average computational time for each instance set. Further, we analyze the ratio between the computational times without and with our graph reduction technique, from here on referred to as *speed-up factor*. As can be seen, the overall complexity of the instances increases exponentially with the number of cross-aisles. The computational times remain stable for each instance set, and settings with up to five cross-aisles that are relevant in practice can be solved in less than a few seconds. Instances with seven cross-aisles already take a few minutes of computational time, and instances with eight cross-aisles do no longer show stable computational times as some instances cannot be solved because of memory limitations. Without our graph reduction technique, instances with seven cross-aisles already show unstable computational performance. The impact of the speed-up factor varies depending on the instance characteristics. The statistical quantities in Table 3 show that in approximately 50% of the cases, our graph reduction technique reduces computational times by at least 50%. In general, the improvement potential of this graph reduction technique tends to be higher, the higher is the number of cross-aisles.

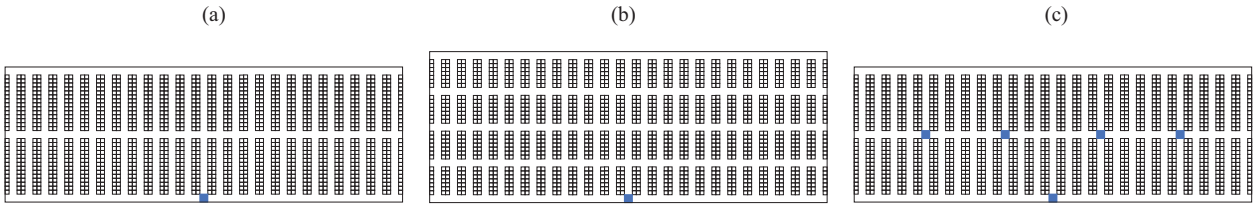
Besides the number of aisles and cross-aisles, the number of consecutively processed picklists and the number of items on each list influence the computational tractability of our algorithm. Furthermore, this tractability heavily depends on the chosen picking policy and on the number of drop-off points in a picking zone. The simplest real-world configuration comprises a static batching (SB), a single drop-off point (SD), and no cartless subtours (NS) on a layout with three cross-aisles and one drop-off point. The most complex real-world configuration comprises a dynamic batching (DS), multiple drop-off points (MD), and cartless subtours (CS) on a layout with three cross-aisles and five drop-off points. Figure 11 shows the computational times for these two extreme configurations. Computational times remain stable below 0.1 second for the simple case. For the complex case, computational times increase significantly for large instances but remain rather insensitive to the number of consecutive picklists because of the dominance criterion introduced in Section 3.3.1. In practice, such algorithms are used in a receding-horizon fashion within a 15-minute interval.

Figure 11. (Color online) Picking Zone Layouts for Three Real-World Cases, by Increasing Order of Complexity



Notes. (a) Layout 1, (SB/SD/NS). (b) Layout 3, (DB/MD/CS).

Figure 12. (Color online) Computational Times Dependent on the Number of Consecutive Lists and Items Per List



Notes. (a) Layout 1 (Zalando). (b) Layout 2 (Hermes). (c) Layout 3 (Amazon).

This corresponds to a maximum number of four to five consecutive picklists with up to 35 items (see Section 4.2). For these real-world configurations shown by the highlighted area in Figure 11, we achieve computational times below 90 seconds even for the worst scenario.

Concluding, our algorithm yields both new best known results for existing benchmarks and a generic framework that is amenable to a real-time implementation for a multitude of real-world configurations with respect to picking zone layouts and picking policies. In the following, we extend our experiments by using the bidirectional LGA as it provides the best and a robust performance on instance sizes relevant in practice.

4.2. Impact of Different Picking Policies and Picking Zone Layouts

The three real-world applications discussed in Section 2.2 are similar with respect to the size and throughput of the warehouse. Each comprises roughly 15,000,000 items of SKUs, which is the total storage capacity of the Zalando warehouse. This translates into 400,000 items for a single picking zone. A picking zone has a width of 70–80 m; each aisle or cross-aisle has a width of 1.5 m; a shelf has a depth of 0.75 m, each storing about 300 items. Hence, 1,400 shelves are necessary to store 400,000 items. Whereas these characteristics are similar for all three warehouses, their layouts vary in terms of the number of cross-aisles and drop-off points: Zalando (Figure 12(a)) uses the most conventional layout with a single drop-off point (blue square) and three cross-aisles, at the beginning, in the middle, and at the rear of the picking zone; Hermes (Figure 12(b)) works with a layout having a single drop-off point but uses five cross-aisles to increase the flexibility of the pickers; the Amazon layout (Figure 12(c)) has three cross-aisles but four additional drop-off points in the middle cross-aisle. In the following, we perform a numerical study based on these case studies to derive insights into the optimal design of picking policies and the respective picking zone layouts.

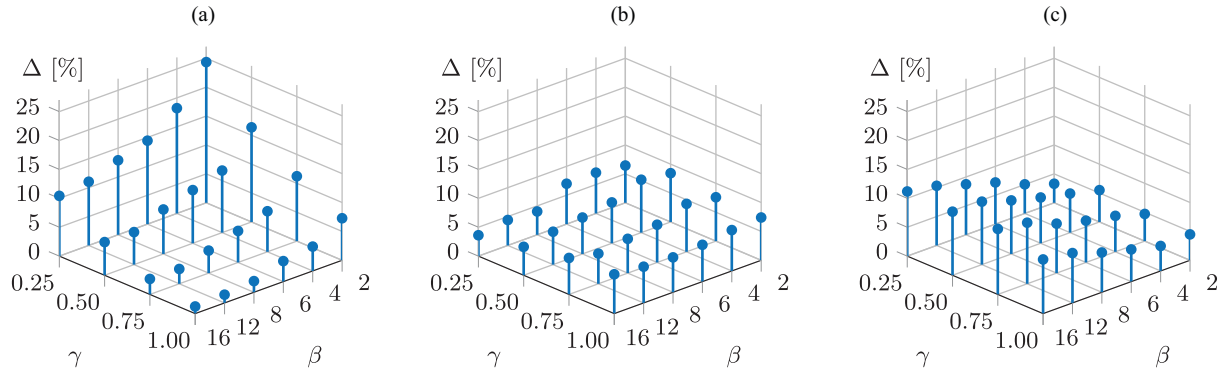
4.2.1. Experimental Design. To allow for an unbiased analysis, we need to work with identical picklist sequences and picking zone sizes. Hence, we cannot use real data from our three industrial cases but generate realistic synthetic instance data.

To ensure equal picking zone sizes that reflect the spacious conditions of our real-world cases, we choose a picking zone width of 75 m that results in a layout with 25 aisles to cover 50 shelf rows, each row having 28 shelves. The width of a shelf is 2 m, and hence we divide each shelve into two picking positions to derive a distance granularity of 1 m. Further, we generate instances with three and five cross-aisles and one to six drop-off points to account for all picking zone settings from our real-world examples and potential variations.

To ensure identical picklist sequences, we generate realistic picklist sequences as detailed in Online Appendix G.2 that mimic a potential upstream batching procedure. Here, we account for varying picklist dispersions, that is, picklists spanning across a varying spatial area within a picking zone, to account for varying batching efficiency. To characterize a picklist dispersion, we use the number of aisles $\beta \in \mathbb{N}$ to the left and right and the share of pick positions $\gamma \in [0,1]$ above and below a picklist's centroid to define the picking zone area in which the list's items are distributed. Small β, γ values denote clustered picklists, whereas large β, γ values denote dispersed picklists. The former means that the previous batching planning stage has successfully clustered all picking positions in a closely confined area, whereas the latter represents a less successful batch plan where picking positions are widely scattered throughout the picking zone. Because all layouts have the same number of aisles and shelves per aisle, this technique allows us to use the derived picklists on each layout and hence to produce comparable unbiased results.

We perform a full factorial design with respect to the picking policy characteristics (Table 1). To this end, we refer to a picking policy with three identifiers, where the first (SB, DB) indicates whether batching is static (SB) or dynamic (DB), the second (SD, MD) indicates whether a single (SD) or multiple (MD) drop-off points are used, and the third (NS, CS) indicates whether cartless subtours are allowed (CS) or not (NS). Moreover, we consider a varying cart capacity κ

Figure 13. (Color online) Average TPTR (Δ) for Picking Policy Flexibility Levers



Notes. (a) SB/MD/NS TPTR for $\kappa = 2$, 5 cross-aisles, and six drop-off points. (b) DB/SD/NS TPTR for $\kappa = 4$, three cross-aisles, and one drop-off point. (c) SB/SD/CS TPTR for $\kappa = 2$, three cross-aisles, and one drop-off point.

between two (Amazon, Zalando) and four (Hermes) bins on a cart that influences the number of parallel processed picklists.

For each resulting setting, we analyze a representative set of 50 instances with five consecutive picklists of 35 items each. This corresponds to a 15-minute planning interval in a receding-horizon setting, which is often used in practice. To evaluate the performance of the different picking policies and picking zone layouts, we analyze the average total picking time reduction (TPTR) related to a picking zone layout with a single depot and the most elementary picking policy (SB/SD/NS). We calculated the TPTR by first comparing the results on a per-instance basis and then computing an average value.

The remainder of this section summarizes our main findings for selected settings, whereas Online Appendix I contains detailed results.

Two comments on this experimental design are in order. First, we note that we focus our analyses on the impact of picking policies and the respective picking zone layouts for varying picklist dispersions and cart capacities. We do not analyze the benefit of increasing the cart capacity explicitly as such an analysis would require a different experimental setup that focuses on a larger time horizon. We refer the interested reader to Online Appendix H for a detailed discussion and preliminary analyses of this impact. Second, although extensive, these studies are not exhaustive because considering layouts with an even number of cross-aisles or further parameter studies may reveal additional benefits. However, we leave these studies for further research as they would require considerable space to allow for rigorous elaboration and discussion.

4.2.2. Picking Policies. Figure 13 focuses on the improvement potential of each picking policy depending on the picklists' dispersion. Here, Figure 13(a) addresses the impact of multiple drop-off points and shows the average TPTR when using an SB/MD/NS policy with $\kappa = 2$ on a five-cross-aisle layout with six drop-off points. We observe a maximum improvement of 24.3% for clustered picklists with the smallest β, γ. For increasing β, γ, the TPTR may be significantly lower, in the worst case as little as 1%. We see similar trends but different amplitudes for settings with $\kappa \in \{3,4\}$ and layouts with three cross-aisles. The maximum TPTR for three-cross-aisle layouts remains 3%–4% below the maximum TPTR for five-cross-aisle layouts. Settings with $\kappa \in \{3,4\}$ show amplitudes that are 7%–8% below the $\kappa = 2$ setting.

Result 1. Multiple drop-off points can yield TPTRs up to 24% on average, given clustered picklists. For dispersed picklists, these TPTRs may become negligible.

Figure 13(b) explores the impact of dynamic batching and shows the average TPTR when using a DB/SD/NS policy with $\kappa = 4$ on a three-cross-aisle layout with one drop-off point. Here, we observe maximum TPTRs of 9% and minimum TPTRs of 3%. Similar patterns occur for varying κ and three- as well as five-cross-aisle layouts, with slightly higher amplitudes on three-cross-aisle layouts. Although settings with clustered picklists still show greater improvements than settings with dispersed picklists, we do not observe the steep systemic decrease seen for multiple drop-off points. In fact, we observe the highest improvement potential for $\gamma = 0.5$ and $\beta \in \{2,4\}$.

Result 2. Dynamic batching can yield TPTRs up to 9.2% on average for clustered picklists. For dispersed picklists, the TPTRs may reduce to 3%.

Figure 13(c) focuses on the impact of cartless subtours and shows the average TPTR when using an SB/SD/CS policy with $\kappa = 2$ on a three-cross-aisle layout with one drop-off point. Interestingly, we observe an

inverse pattern with the highest TPTRs being realized for dispersed picklists. Independent of κ or the layout's number of cross-aisles, we observe similar patterns with only slight amplitude variations of roughly 1%.

Result 3. Cartless subtours can yield TPTRs up to 11.3% on average for dispersed picklists. For clustered picklists, the improvement may reduce to 3%.

Figure 14 further details the impact of multiple drop-off points by showing the TPTR for selected levels of picklist dispersion dependent on the number of drop-off points available. As can be seen, the marginal utility of adding an extra drop-off point decreases significantly for settings beyond three drop-off points. This effect is independent of κ or the cross-aisle layout, and we note that the marginal utility decrease is slightly lower for clustered picklists than for dispersed picklists.

Result 4. The deployment of multiple drop-off points shows decreasing marginal utilities for large numbers of drop-off points such that the positive effect of additional drop-off points quickly diminishes the more drop-off points are added.

Figure 15 shows the TPTR when utilizing all picking policy characteristics (DB/MD/CS) for $\kappa = 2$ on a five-cross-aisle layout. We observe a maximum average TPTR of 31% for maximal clustered picklists and decreasing TPTR for more dispersed picklists, yielding a minimum TPTR of 12%. Similar patterns arise for three-cross-aisle layouts and settings with varying κ. Although minimum average improvements remain stable between 11% and 12% for all settings, maximum TPTRs vary between 31% and 21%.

Result 5. Exploiting all picking policy flexibility levers can yield TPTRs up to 31% for clustered picklists. For dispersed picklists, the TPTR may reduce to 11%.

4.2.3. Synthesis. Our observations allow us to synthesize the following insights. First, we note that the

Figure 14. (Color online) Average SB/MD/NS TPTR for $\kappa = 2$ with Five Cross-Aisles for Different Levels of Picklist Dispersion (β,γ) Dependent on the Number of Drop-off Points

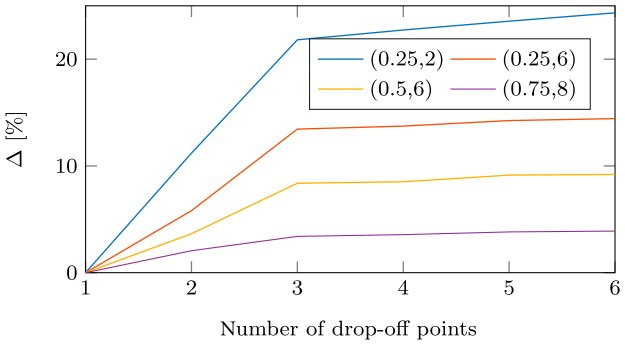
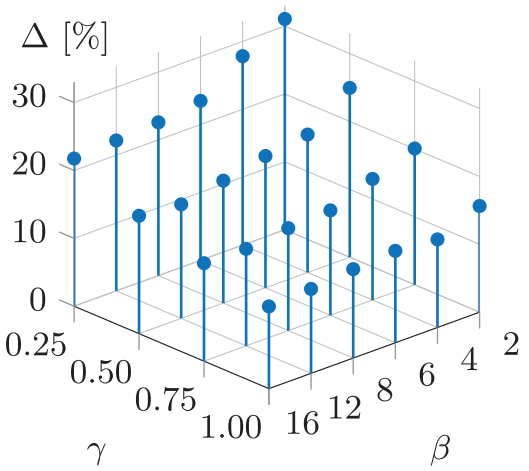


Figure 15. (Color online) Average DB/MD/CS TPTR with $\kappa = 2$ with Five Cross-Aisles and Six Drop-off Points



TPTR potential of a specific picking policy depends highly on the picklist dispersion, that is, the efficiency of an upstream batching: the more efficient the upstream batching, that is, the more clustered the picklists, the larger is the benefit of multiple drop-off points. This allows for a significantly improved routing and early dropping-off of asynchronously finished picklists. For dispersed picklists, it is less likely that the completion time of picklists hugely vary, that is, that picklists can be dropped off early, and the benefit of multiple drop-offs decreases significantly for inefficient upstream batching. Independent of the batching efficiency, we observe decreasing marginal utilities when increasing the number of deployed drop-off points beyond three drop-off points.

Second, we observe a reversed effect for cartless subtours, which yield the greatest TPTRs in settings with inefficient batching. In these cases, the picking routes span over a large part of the picking zone, which naturally leads to more slopes and subtours that allow for savings, whereas clustered picklists reduce the gains of cartless subtours.

Third, the TPTR potential of dynamic batching appears to be relatively insensitive to the picklist dispersion as changing the order of active picklists bears a TPTR potential in various settings.

Focusing on the overall TPTR potential, the impact of the different levers is additive, that is, by utilizing all three levers, we reach a TPTR potential similar to the sum of the single TPTR potentials. We do not observe distinct reinforcing effects; across all settings, the overall TPTR potential and the sum of the single TPTR potentials show positive or negative deviations below 2%.

4.2.4. Picking Zone Layout. Besides the picking policies, changing the picking zone layout with respect

Table 4. TPTR Between Five and Three-Cross-Aisle Layouts for Varying Picklist Distributions

β	2				4				6				8				12				16			
γ	0.25	0.50	0.75	1.00	0.25	0.50	0.75	1.00	0.25	0.50	0.75	1.00	0.25	0.50	0.75	1.00	0.25	0.50	0.75	1.00	0.25	0.50	0.75	1.00
Max	6.3	6.4	6.3	−0.7	10.5	8.3	10.3	3.3	14.9	10.6	14.2	7.9	15.1	14.1	16.9	9.7	17.5	17.0	21.5	14.1	18.2	17.8	23.2	16.0
Q3	4.7	5.4	4.6	−2.3	8.8	6.8	8.5	1.5	12.3	9.0	12.3	6.4	12.9	11.6	15.6	8.9	14.9	14.6	19.7	12.9	15.6	15.9	20.6	14.2
Q2	4.0	4.3	3.5	−2.9	7.2	5.8	6.9	0.6	11.0	8.2	10.3	3.9	11.9	10.4	13.1	6.0	13.6	13.8	17.8	10.1	15.0	15.4	19.7	12.8
Q1	2.2	3.1	2.8	−4.1	6.1	4.7	6.2	−0.5	10.4	7.6	9.5	2.8	11.3	9.0	12.7	5.0	12.8	12.5	16.3	8.7	13.8	14.3	18.5	10.6
Min	0.3	2.1	0.7	−5.1	4.5	3.2	5.0	−1.9	9.1	6.6	7.4	1.8	9.8	7.7	11.7	3.9	11.2	12.1	15.5	7.4	12.4	13.3	17.0	9.1
Mean	3.6	4.2	3.6	−3.1	7.5	5.8	7.3	0.6	11.4	8.4	10.8	4.3	12.1	10.5	13.9	6.6	13.9	13.9	18.0	10.5	14.9	15.3	19.7	12.6

Note. This table characterizes the TPTR distribution when switching from a three-cross-aisle to a five-cross-aisle layout.

to the number of cross-aisles may impact the picking performance. Table 4 shows for all levels of picklist dispersion the range of average TPTRs over all picking policies and cart capacities κ when switching from a three-cross-aisle layout to a five-cross-aisle layout. We observe greater reductions for dispersed picklists with a maximum reduction of 23.2% for $\beta = 16$ and $\gamma = 0.75$. For clustered picklists, we observe significantly lower reductions. For high γ and low β, we even observe increased total picking times in some settings. This effect highlights that additional cross-aisles increase the flexibility of the picker to shortcut into adjacent aisles but add unused space to the warehouse and increase the lengths of the picking aisles. Particularly for clustered picklists that require only a few aisles to be traversed, additional cross-aisles constitute a significant share of the overall distance, which may worsen the picking time. Hence, there exists a general trade-off between different design and operational objectives; we cannot conclude that more cross-aisles automatically increase picking performance. This trade-off has been explored in great detail for traditional (single-depot) settings, for example, by Roodbergen and De Koster (2001) as well as Roodbergen et al. (2008).

Result 6. Increasing the number of cross-aisles significantly reduces the picking time for settings with dispersed picklists. For clustered picklists, the reduction is much smaller and increasing the number of cross-aisles may even lead to a loss of picking performance.

5. Conclusions

With the rapid growth in e-commerce, picker routing and its associated policies have become crucial determinants of profitability and competitiveness in warehouse operations. We have presented the first generic and exact algorithmic framework for the GOPP, which constitutes a new state of the art for several known (sub)problem variants and for the generic case. Our algorithm is scalable and amenable to real-time applications, requiring computational times of less than two seconds for realistic warehouse layouts and picking policies. Using this algorithm, we have analyzed the impact of several picking policies and warehouse layouts on the overall performance of a picker.

Although not exhaustive in all design aspects of picking zones, our results allow the derivation of several meaningful managerial insights, identifying trends that may open the field for further research. We have shown that employing enhanced picking policies may reduce the average total picker walking time by up to 20%–30% depending on the picking zone layout and cart capacity used, which provides the potential to increase overall flexibility or throughput. Utilizing multiple drop-off points is highly beneficial in case of clustered picklists that result from an efficient upstream batching procedure. Its impact becomes negligible for scattered picklists, that is, in the absence of an efficient upstream batching. Conversely, cartless subtours appear to be particularly beneficial in the presence of scattered picklists and may partly compensate for inefficient upstream batching but show only little impact for settings with clustered picklists. Dynamic batching allows for improvements independently of the upstream batching's efficiency and the resulting picklist dispersion. Increasing the number of cross-aisles in a picking zone layout can improve or deteriorate the picking times because deciding on the number of cross-aisles amounts to making a trade-off between routing flexibility and walking distances. Accordingly, using layouts with larger numbers of cross-aisles appears to be particularly beneficial for dispersed picklists but may worsen picking times when an efficient upstream batching generates significantly clustered picklists.

Two final comments in relation with these insights are in order. First, the impact of cartless subtours may be significantly higher or lower based on the ratio between a picker's speed with and without a cart. The picker speeds used in this work are often observed in practice, which means that our results provide a good baseline; but they should be reexamined if achievable speeds differ significantly. Second, explicitly analyzing the benefit of varying cart capacities may reveal additional interesting insights. Here, we want to emphasize that our results constitute only a first step into this field of research. Extending our framework to provide in-depth analyses of different objectives or

integrated batching and order picking, which has been so far mostly studied for rather theoretical warehouse settings, may yield interesting insights for both academics and practitioners.

Acknowledgments

The authors thank three anonymous referees, an anonymous associate editor, and the department editor Chung Piaw Teo for their valuable feedback, which helped to improve this paper significantly. The authors thank Konrad Stephan for his feedback and proofreading the paper.

References

Azadeh K, De Koster RMB, Roy D (2018) Robotized and automated warehouse systems: Review and recent developments. *Transportation Sci.* 53(4):917–940.

Boysen N, De Koster RBM, Weidinger F (2019a) Warehousing in the e-commerce era: A survey. *Eur. J. Oper. Res.* 277(2): 396–411.

Boysen N, Fedtke S, Weidinger F (2018) Optimizing automated sorting in warehouses: The minimum order spread sequencing problem. *Eur. J. Oper. Res.* 270(1):386–400.

Boysen N, Stephan K, Weidinger F (2019b) Manual order consolidation with put walls: The batched order bin sequencing problem. *EURO J. Transportation Logist.* 8(2):169–193.

Bozer YA, Kile JW (2008) Order batching in walk-and-pick order picking systems. *Internat. J. Production Res.* 46(7):1887–1909.

Cambazard H, Catusse N (2018) Fixed-parameter algorithms for rectilinear Steiner tree and rectilinear traveling salesman problem in the plane. *Eur. J. Oper. Res.* 270(2):419–429.

Chen F, Wang H, Qi C, Xie Y (2013) An ant colony optimization routing algorithm for two order pickers with congestion consideration. *Comput. Indust. Engrg.* 66(1):77–85.

Cui R, Zhang DJ, Bassamboo A (2019) Learning from inventory availability information: Evidence from field experiments on Amazon. *Management Sci.* 65(3):1216–1235.

Daniels RL, Rummel JL, Schantz R (1998) A model for warehouse order picking. *Eur. J. Oper. Res.* 105(1):1–17.

De Koster RBM, Van der Poort ES (1998) Routing orderpickers in a warehouse: A comparison between optimal and heuristic solutions. *IIE Trans.* 30(5):469–480.

De Koster RBM, Le-Duc T, Roodbergen KJ (2007) Design and control of warehouse order picking: A literature review. *Eur. J. Oper. Res.* 182(2):481–501.

Gallien J, Weber T (2010) To wave or not to wave? Order release policies for warehouses with an automated sorter. *Manufacturing Service Oper. Management* 12(4):642–662.

Goetschalckx M, Ratliff HD (1988) An efficient algorithm to cluster order picking items in a wide aisle. *Engrg. Costs Production Econom.* 13(4):263–271.

Gue KR, Meller RD, Skufca JD (2006) The effects of pick density on order picking areas with narrow aisles. *IIE Trans.* 38(10):859–868.

Hall RW (1993) Distance approximations for routing manual pickers in a warehouse. *IIE Trans.* 25(4):76–87.

Hanan M (1966) On Steiner's problem with rectilinear distance. *SIAM J. Appl. Math.* 14(2):255–265.

Henn S, Wäscher G (2012) Tabu search heuristics for the order batching problem in manual order picking systems. *Euro. J. Oper. Res.* 222(3):484–494.

Hong S, Johnson AL, Peters BA (2012a) Batch picking in narrow-aisle order picking systems with consideration for picker blocking. *Euro. J. Oper. Res.* 221(3):557–570.

Hong S, Johnson AL, Peters BA (2012b) Large-scale order batching in parallel-aisle picking systems. *IIE Trans.* 44(2):88–106.

Pansart L, Catusse N, Cambazard H (2018) Exact algorithms for the picking problem. *Comput. Oper. Res.* 100:117–127.

Petersen CG (1997) An evaluation of order picking routing policies. *Internat. J. Oper. Production Management* 17(11):1098–1111.

Ratliff HD, Rosenthal AS (1983) Order-picking in a rectangular warehouse: A solvable case of the traveling salesman problem. *Oper. Res.* 31(3):507–521.

Roodbergen KJ, De Koster RBM (2001) Routing order pickers in a warehouse with a middle aisle. *Eur. J. Oper. Res.* 133(1): 32–43.

Roodbergen KJ, Sharp GP, Vis IFA (2008) Designing the layout structure of manual order picking areas in warehouses. *IIE Trans.* 40(11):1032–1045.

Theys C, Bräysy O, Dullaert W, Raa B (2010) Using a TSP heuristic for routing order pickers in warehouses. *Eur. J. Oper. Res.* 200(3): 755–763.

Weidinger F (2018) Picker routing in rectangular mixed shelves warehouses. *Comput. Oper. Res.* 95:139–150.

Weidinger F, Boysen N (2018) Scattered storage: How to distribute stock keeping units all around a mixed-shelves warehouse. *Transportation Sci.* 52(6):1412–1427.

Weidinger F, Boysen N, Schneider M (2019) Picker routing in the mixed-shelves warehouses of e-commerce retailers. *Eur. J. Oper. Res.* 274(2):501–515.

Zulj I, Kramer S, Schneider M (2018) A hybrid of adaptive large neighborhood search and tabu search for the order-batching problem. *Eur. J. Oper. Res.* 264(2):653–664.